21CSC201J
DATA STRUCTURES AND ALGORITHMS

UNIT 1 - Introduction

Prepared by

Dr L Josephine Usha
Dr P Shunmuga Sundari
Dr R Deepalakshmi

Topics to be covered

Programming in C – Primitive data types, Structures, Self-referential structures, Pointers and structures, Dynamic memory allocation, Matrix multiplication; Data Structure – Definition, Types, ADT, Operations; Mathematical notations - Big O, Omega and Theta , Complexity – Time, Space, Trade off.

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

INTRODUCTION
BASIC TERMINOLOGY

Introduction

- Data Structure can be defined as the group of data elements which provides an efficient way of storing and organizing data in the computer so that it can be used efficiently.
- Some examples of Data Structures are arrays, Linked List, Stack, Queue, etc.
- Data Structures are widely used in almost every aspect of Computer Science i.e. Operating System, Compiler Design, Artificial intelligence, Graphics and many more.

Basic Terminology

- Data are values or sets of values
- A data item refers to a single unit of values. Data items are divided into subitems are called group items.
  - For example, an employee's name may be divided into three subitems first name, middle name and last name but the social security number would treated as a single name.
- Collections of data are organized into a hierarchy of fields, records and files.

Basic Terminology (Cont..)

- Record can be defined as the collection of various data items.
  - For example, if we talk about the student entity, then its name, address, course and marks can be grouped together to form the record for the student.
- Record is classified according to length. A file can have fixed length records or variable length records.
- In fixed length records, all the records contain the same data items with the same amount of space assigned to each data items.
- In variable length records, file records may contain different lengths. Variable length records have a minimum and a maximum length.
  - For example, student records have variable length, since different students take different numbers of courses

Basic Terminology (Cont..)

- A File is a collection of various records of one type of entity.
  - For example, if there are 60 employees in the class, then there will be 20 records in the related file where each record contains the data about each employee.
- An entity represents the class of certain objects. It contains various attributes. Each attribute represents the particular property of that entity.
  - For example, consider an employee of an organization:

Attributes:	Name	Age	Sex	Social Security Number
Values:	JOHN	34	M	134-24-5533

- Field is a single elementary unit of information representing the attribute of an entity.

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

Programming in C

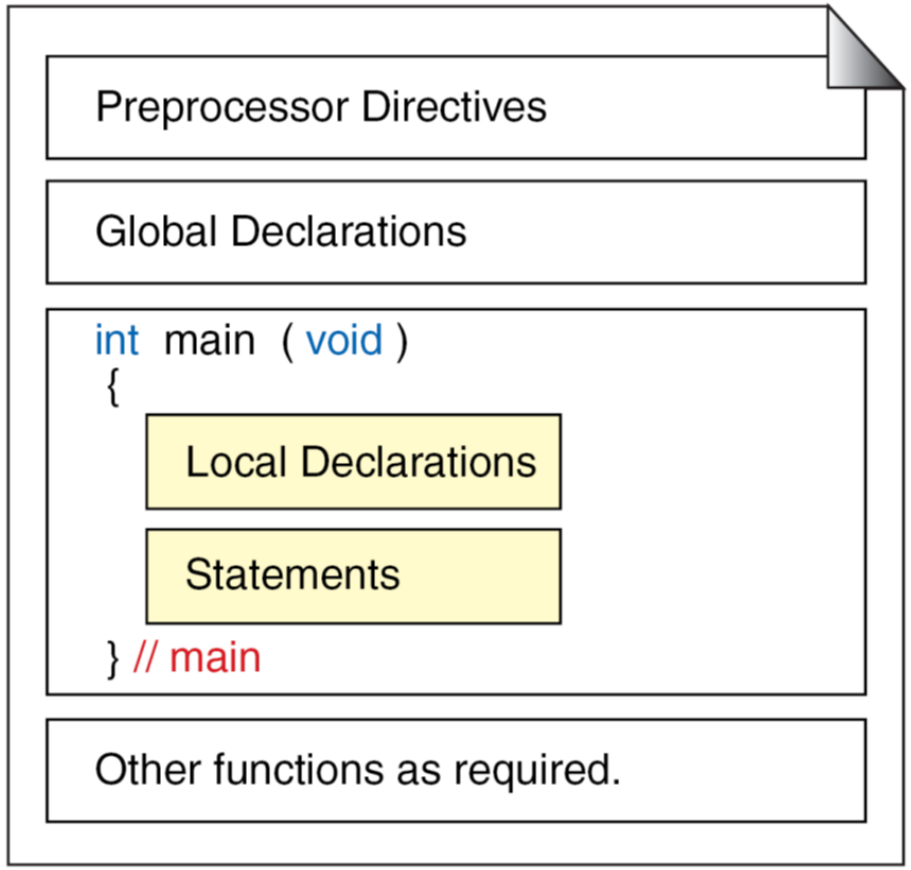
Programming in C – Introduction

- C is a programming language developed at AT & T Bell Laboratories of USA in 1972, designed and written by "Dennis Ritchie".
- C is highly portable i.e., software written for one computer can be run on another computer.
- C is a structured programming language.
- An important feature of C is its ability to extend itself.
- Basically it is a collection of functions.

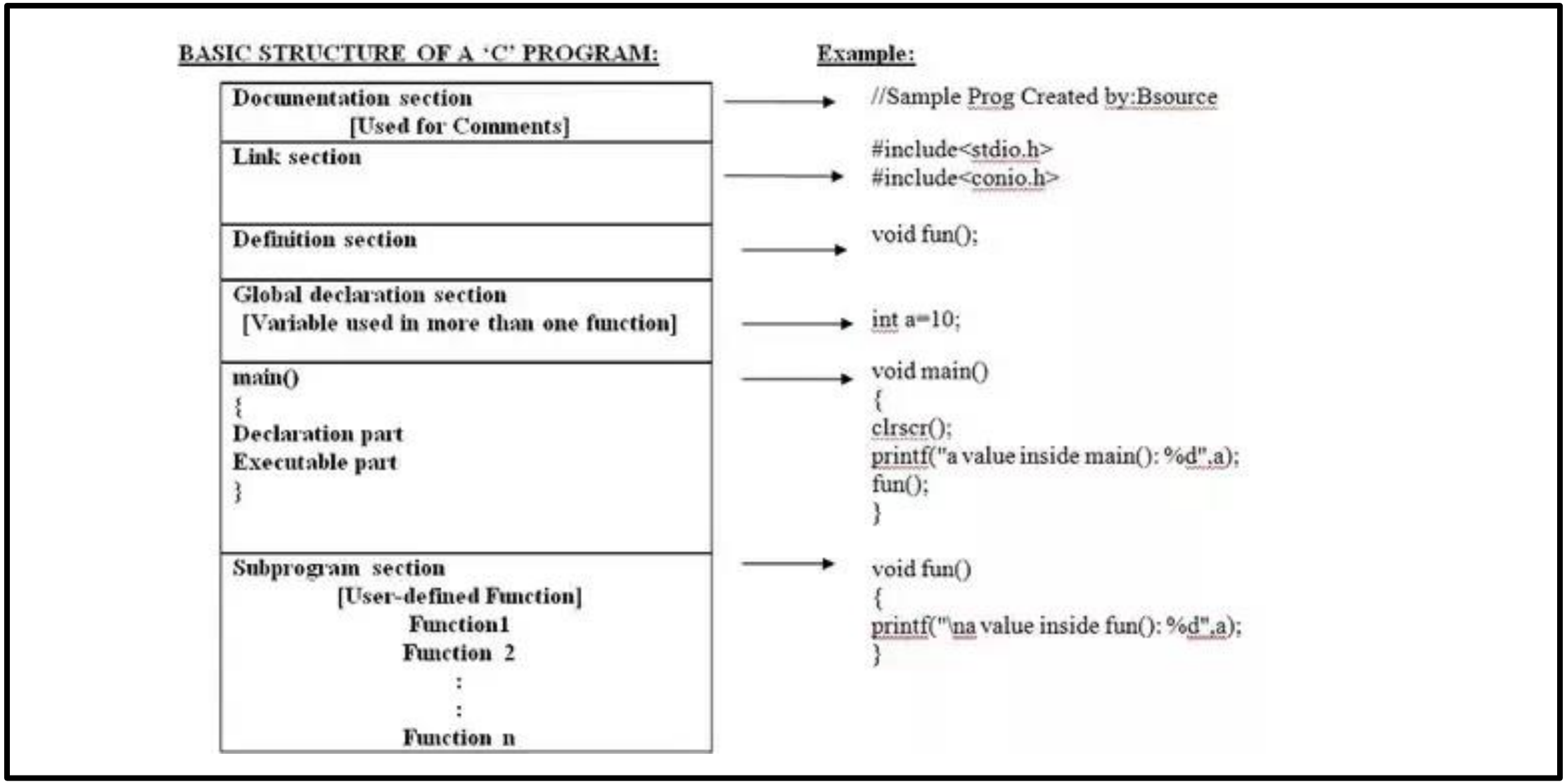
Programming in C – History of C

- 1960: ALGOL (ALGOrithmic Language)
- 1967: BCPL (Basic Combined Programming Language)
- 1970: B programming language (typeless)
- 1972: C: BCPL plus B with types
- 1978: Kernighan + Ritchie standard for C
- 1989: ANSI standard for C

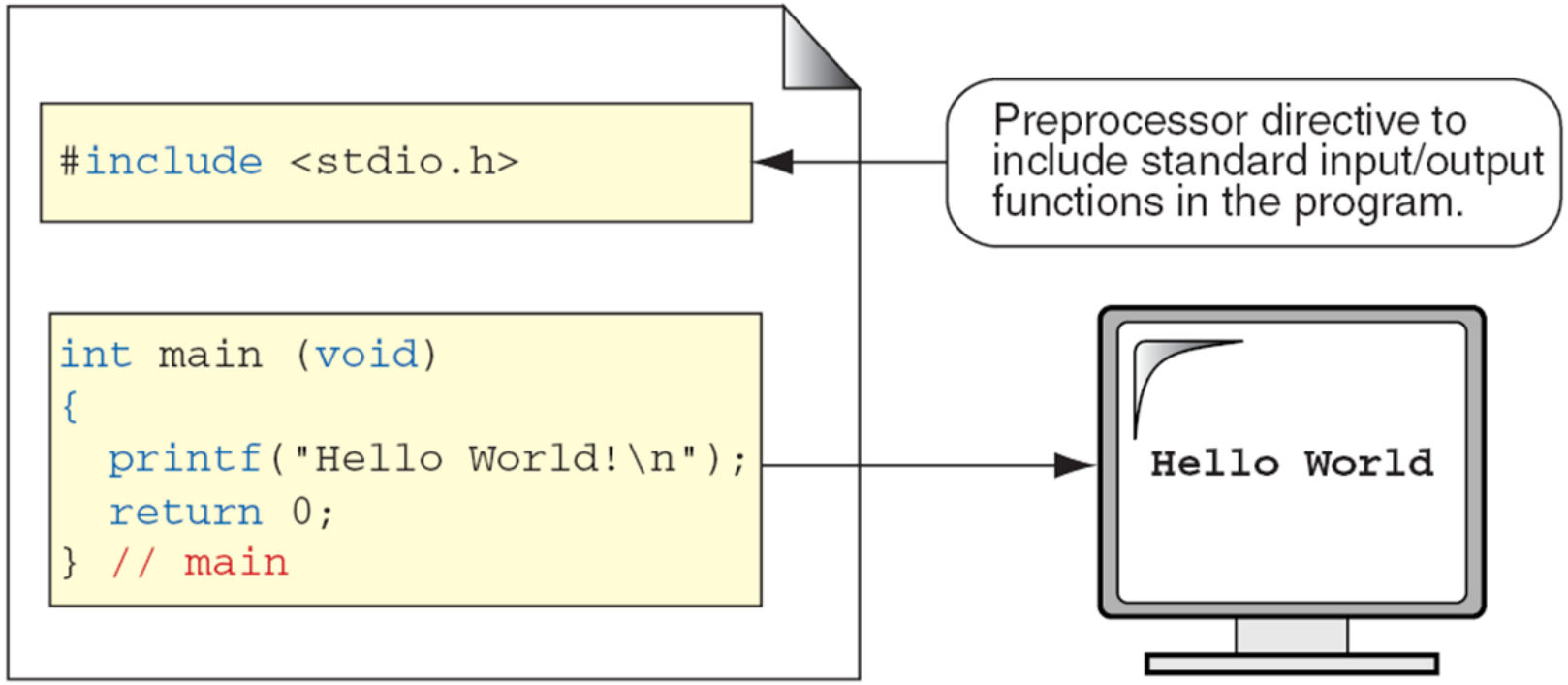
Programming in C – Structure of C Program



Programming in C – Structure of C Program



Programming in C – Example



Explanation

Elements	Descriptions
/* Comments */	Comments are a way of explaining what makes a program. Comments are ignored by the compiler and used by others to understand the code.
#include<stdio.h>	This is a preprocessor command that notify the compiler to include the header file stdio.h in the program before compiling the source-code.
int/void main()	int/void is a return value.
main()	The main() is the main function where program execution begins. Every C program must contain only one main function.
Braces	Two curly brackets "{…}" are used to group all
printf()	It is a function in C, which prints text on the screen.

Programming in C – Comments

SRM INSTITUTE OF SCIENCE & TECHNOLOGY (Deemed to be University u/s 3 of UGC Act, 1956)

Example of Block Comments

```
/* This is a block comment that
   covers two lines.                    */


/*
** It is a very common style to put the opening token
** on a line by itself, followed by the documentation
** and then the closing token on a separate line. Some
** programmers also like to put asterisks at the beginning
** of each line to clearly mark the comment.
*/
```

Programming in C – Comments

Example of Line Comments

```
// This is a whole line comment

a = 5;              // This is a partial line comment
```

Programming in C – Identifiers

- One feature present in all computer languages is the identifier.
- Identifiers allow us to name data and other objects in the program. Each identified object in the computer is stored at a unique address.
- Rules for Identifiers:

1. First character must be alphabetic character or underscore.
2. Must consist only of alphabetic characters, digits, or underscores.
3. First 63 characters of an identifier are significant.
4. Cannot duplicate a keyword.

Note **C is a case-sensitive language.**

Programming in C – Identifiers

- Examples of Valid and Invalid identifiers:

Valid Names		Invalid Name	
a	// Valid but poor style	$sum	// $ is illegal
student_name		2names	// First char digit
_aSystemName		sum-salary	// Contains hyphen
_Bool	// Boolean System id	stdnt Nmbr	// Contains spaces
INT_MIN	// System Defined Value	int	// Keyword

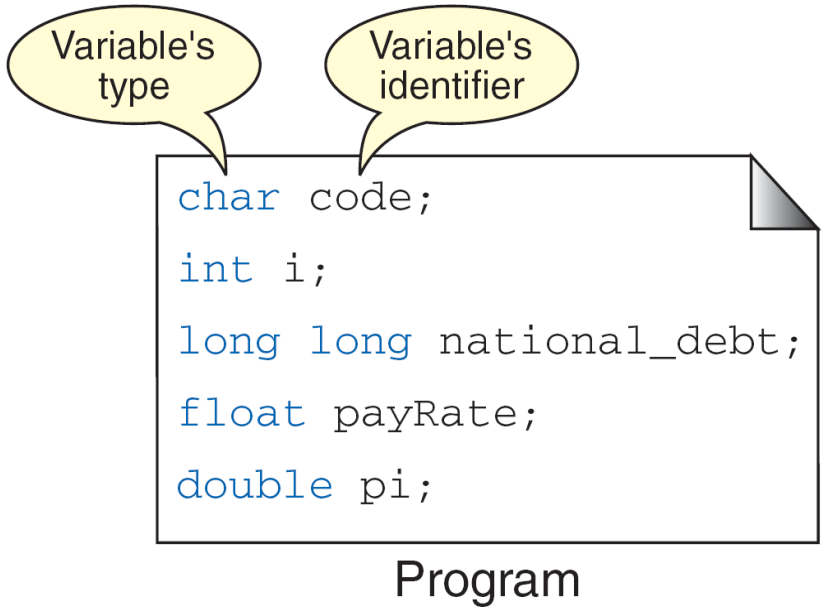
Programming in C – Keywords

- Keywords are nothing but system defined identifiers.
- Keywords are reserved words of the language.
- They have specific meaning in the language and cannot be used by the programmer as variable or constant names
- 32 Keywords in C Programming

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Programming in C – Variables

- Variables are named memory locations that have a type, such as integer or character, which is inherited from their type.
- The type determines the values that a variable may contain and the operations that may be used with its values.



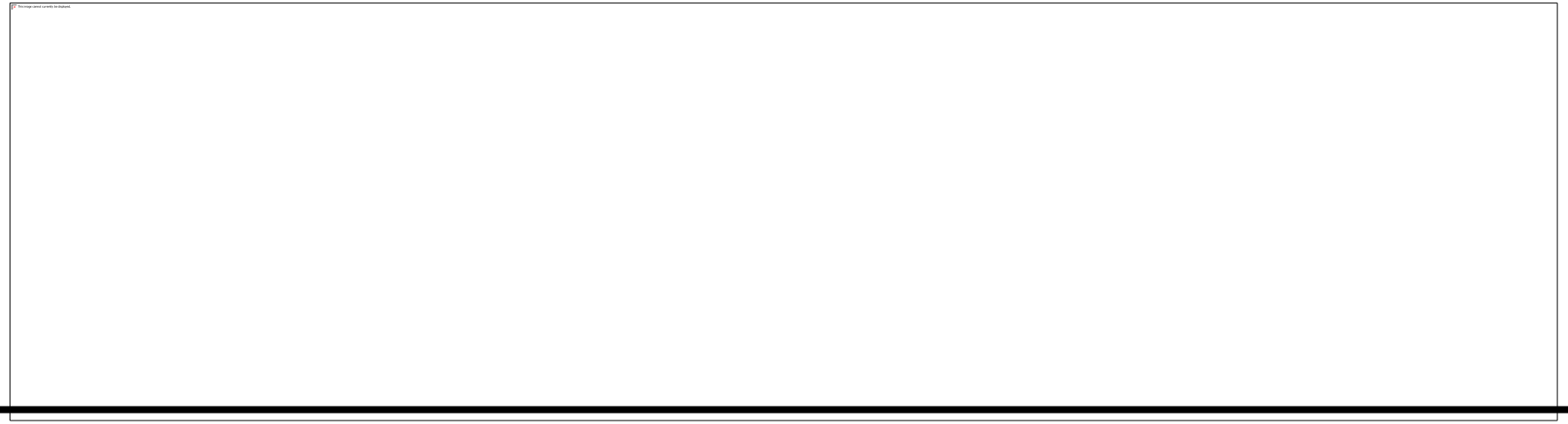
Programming in C – Variable Declarations

SRM INSTITUTE OF SCIENCE & TECHNOLOGY (Deemed to be University u/s 3 of UGC Act, 1956)

Examples of Variable Declarations

```
bool   fact;
short  maxItems;              // Word separator: Capital
long   long national_debt;    // Word separator: underscore
float  payRate;               // Word separator: Capital
double tax;
```

Examples of Variable Initialization



Programming in C – Constants

- Constants are data values that cannot be changed during the execution of a program.
  - eg. const double PI = 3.14
  - Here, PI is a constant. Basically what it means is that, PI and 3.14 is same for this program.
  - Like variables, constants have a type.

Integer constants

- A integer constant is a numeric constant (associated with number) without any fractional or exponential part. There are three types of integer constants in C programming:
  - decimal constant(base 10)
  - octal constant(base 8)
  - hexadecimal constant(base 16)

Programming in C – Constants

Floating-point constants

- A floating point constant is a numeric constant that has either a fractional form or an exponent form.
  - For example: 2.0,0.0000234,-0.22E-5

Character constants

- A character constant is a constant which uses single quotation around characters.
  - For example: 'a', 'l', 'm', 'F'

String constants

- String constants are the constants which are enclosed in a pair of double-quote marks.
  - For example: "good" ,"x","Earth is round\n"

Programming in C – Escape Sequences

- Sometimes, it is necessary to use characters which cannot be typed or has special meaning in C programming.
- For example: newline(enter), tab, question mark etc. In order to use these characters, escape sequence is used.

Sequences	Character
\b	Backspace
\f	Form feed
\n	Newline
\r	Return
\t	Horizontal tab
\v	Vertical tab
\\	Backslash
\'	Single quotation mark
\"	Double quotation mark
\?	Question mark
\0	Null character

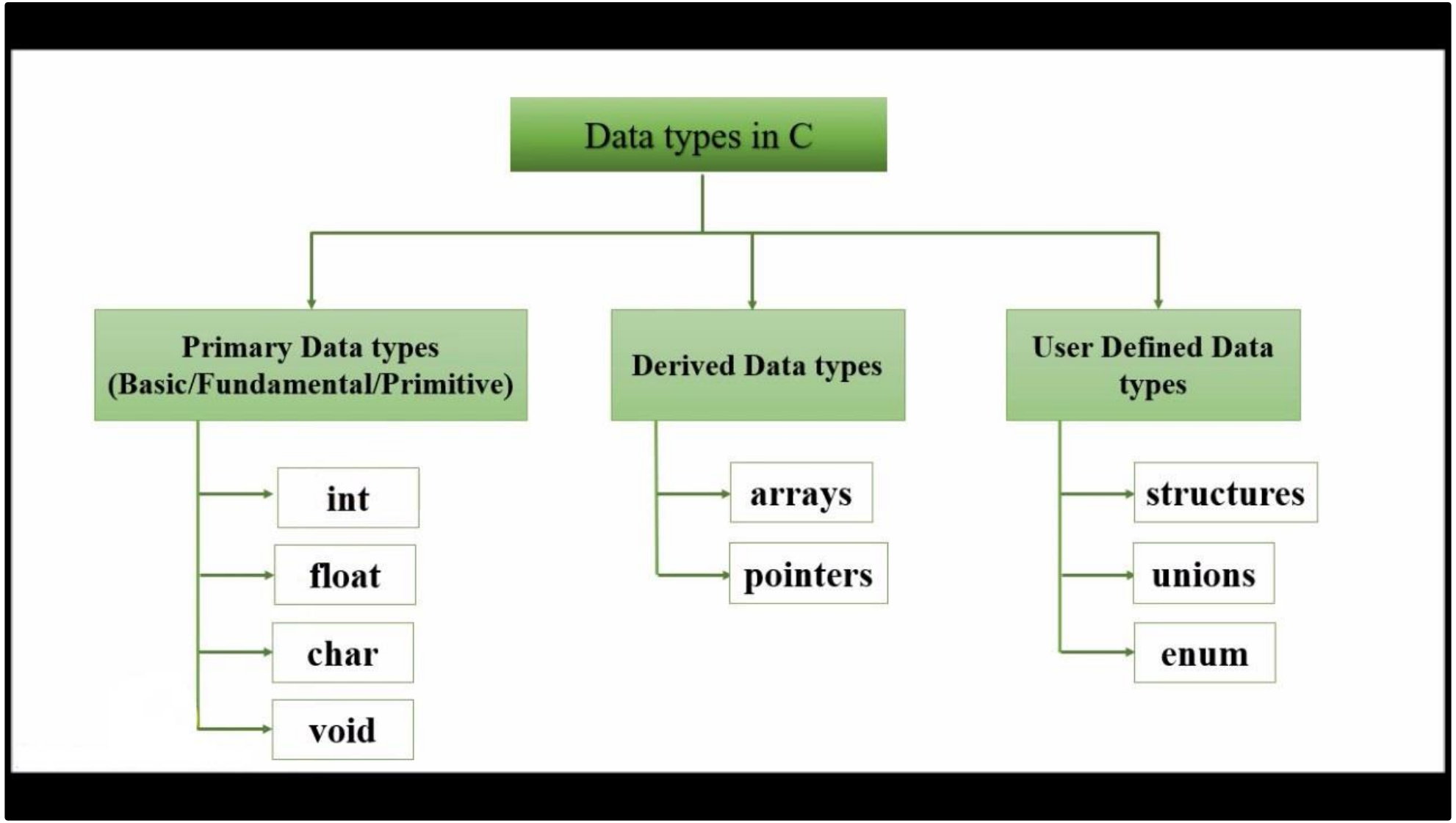
SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

Data Types in C

Programming in C – Data Types

- Each variable in C has an associated data type.
- It specifies the type of data that the variable can store like integer, character, floating, double, etc.
- Each data type requires different amounts of memory and has some specific operations which can be performed over it.
- The data type is a collection of data with values having fixed values, meaning as well as its characteristics.

Programming in C – Data Types



Programming in C – Primitive Data Types

- A **primitive** is a fundamental **data type** that cannot be broken down into a more simple **data type**
- **They are**
  - **character**
  - **integer**
  - **float**
  - **void**

Primitive Data Types - Primitive Data Types

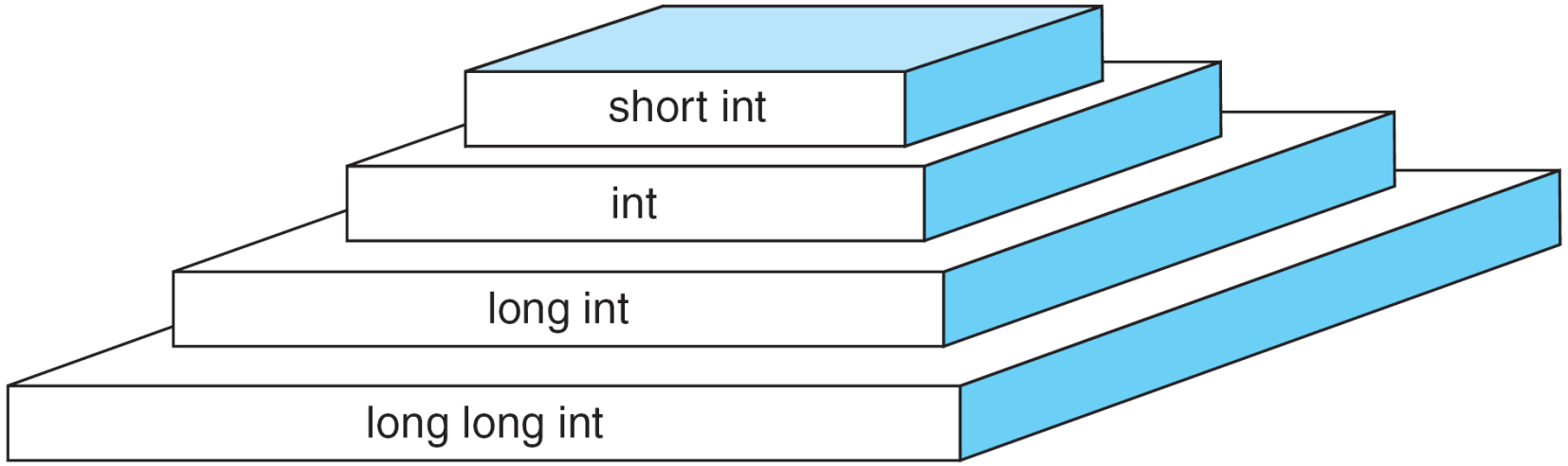
- **Character Data Type**
  - It stores a single character
  - requires a single byte of memory
  - Format specifier - %c
  - Keyword – char
  - Example – char test = ‘h’;

Programming in C – Primitive Data Types

- **Integer Data Type**
- The integer datatype in C is used to store the whole numbers without decimal values. Octal values, hexadecimal values, and decimal values can be stored in int data type in C.
- **Range:** -2,147,483,648 to 2,147,483,647
- **Size:** 4 bytes
- **Format Specifier:** %d

Programming in C – Primitive Data Types

- **Integer Types**



Note

sizeof (short) ≤ sizeof (int) ≤ sizeof (long) ≤ sizeof (long long)

Programming in C – Primitive Data Types

SRM INSTITUTE OF SCIENCE & TECHNOLOGY (Deemed to be University u/s 3 of UGC Act, 1956)

Typical Integer Sizes and Values for Signed Integers

Type	Byte Size	Minimum Value	Maximum Value
short int	2	–32,768	32,767
int	4	–2,147,483,648	2,147,483,647
long int	4	–2,147,483,648	2,147,483,647
long long int	8	–9,223,372,036,854,775,807	9,223,372,036,854,775,806

Programming in C – Primitive Data Types

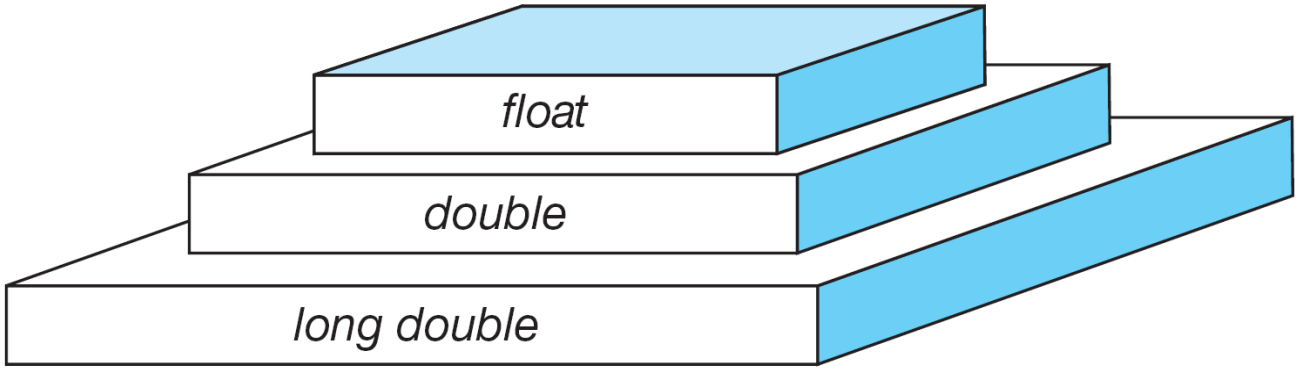
SRM INSTITUTE OF SCIENCE & TECHNOLOGY (Deemed to be University u/s 3 of UGC Act, 1956)

Floating Point Types

- It stores fractional numbers (real numbers)
- Usually requires 4 bytes of memory
- Format specifier - %f
- Keyword – float
- Example – float factor=22.442e2;

Programming in C – Primitive Data Types

- Floating Point Types



Note

sizeof (float) ≤ sizeof (double) ≤ sizeof (long double)

Programming in C – Primitive Data Types

- **Void Data Types**
  - The void data type in C is used to specify that no value is present. It does not provide a result value to its caller.
  - It has no values and no operations. It is used to represent nothing.
  - Void is used in multiple ways as function return type, function arguments as void etc.

Primitive Data Types – void

- is an incomplete type. i.e. "nothing" or "no type".
- It is used in three kinds of situations

Type	Description	Example
Function returns as void	A function with no return value has the return type as void.	void exit (int status);
Function arguments as void	functions which do not accept any parameter.	int rand(void);
Pointers to void	A pointer of type void * represents the address of an object, but not its type	void *malloc(size_t size); returns a pointer to void which can be casted to any data type

Programming in C – Sizeof Data Types

- Size of Data Types in C
- The size of the data types in C is dependent on the size of the architecture, so we cannot define the universal size of the data types.
- For that, the C language provides the sizeof() operator to check the size of the data types.

Programming in C – Sizeof Example

```c
// C Program to print size of different data type in C
#include <stdio.h>

int main()
{
    int size_of_int = sizeof(int);
    int size_of_char = sizeof(char);
    int size_of_float = sizeof(float);
    int size_of_double = sizeof(double);

    printf("The size of int data type : %d\n",
            size_of_int);
    printf("The size of char data type : %d\n",
           size_of_char);
    printf("The size of float data type : %d\n",
           size_of_float);
    printf("The size of double data type : %d",
           size_of_double);

    return 0;
}
```

Output

```
The size of int data type : 4
The size of char data type : 1
The size of float data type : 4
The size of double data type : 8
```

Derived Data Types

- A **derived type** is formed by using one or more basic **types** in combination.
- Using **derived types**, an infinite variety of new **types** can be formed.
- They are
  - arrays
  - pointers

Derived Data Types – arrays

- An array is a collection of similar data types under a single variable name in continuous memory location
- i.e. one or more than one integers can be stored under a single name
- similarly for other data types too
- Example:

```
int new[5] = {1,2,3,4,5};
char name = “Program”;
```

Derived Data Types – pointers

- A pointer is a special variable that stores address of another variable
- Declare a **pointer** before using it to store any variable address.
- Syntax:

```
type *ptr-variable-name;    //declaration
variable-name = & ptr-variable-name;  //assignment
```

- Example:

```
int *p;
int num;
p = &num;
```

User-defined Data Types (UDT)

- **User-defined datatypes** use the built-in **datatypes** and other **user-defined datatypes**
- They are also called as data structures
- They are:
  - ✓ Structure
  - ✓ Union
  - ✓ Enumeration

User-defined Data Types - Structure

- It is a package of variables of different types under a single name.
- Structures are used to represent a record
- The struct statement defines a new data type, with more than one member.
- Keyword – struct
- Syntax:

```
struct variable {
        member(s);
}; variable(s);
```

Example:

```
struct book {
   int book-id = 234;
   char name[20] = "C – Program";
} b1,b2;
```

User-defined Data Types - Union

- A **union** is a special data type that allows to store different data types in the same memory location.
- You can define a union with many members, but only one member can contain a value at any given time.
- Unions provide an efficient way of using the same memory location for multiple-purpose.
- The **union tag** is optional.

User-defined Data Types - Union

- Keyword – union
- Syntax:

```
union variable
{
    member(s);
}; variable(s);
```

- Example:

```
union number
{
  int roll;
  char option;
  float mark;
} n1,n2;
```

- Now, the union variable can store an integer, a floating-point number, or a string of characters.
- It means a single variable, i.e., same memory location, can be used to store multiple types of data.

- The memory occupied by a union will be large enough to hold the largest member of the union
- In the above example, the memory allocated is for float which is the largest data type

User defined Data Types - Enumeration

- Enumeration is a user defined datatype in C language.
- It is used to assign names to the integral constants which makes a program easy to read and maintain.
- Keyword – enum
- Syntax:

  enum variable {constant1, constant2,…}
- Example –

  enum week{Sunday,Monday,Tuesday,Wednesday,…….};

  enum week day;

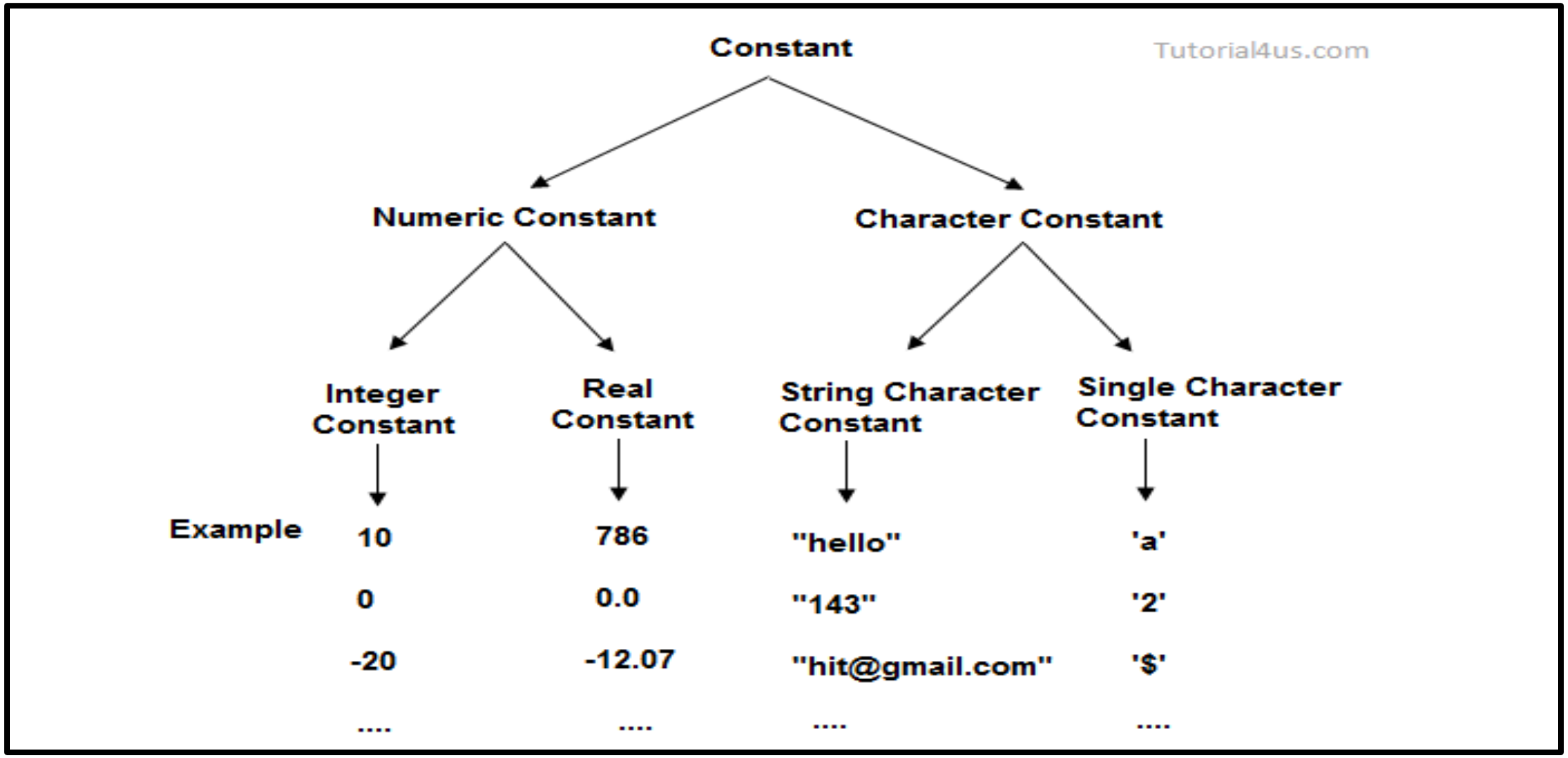
Other Data types…..

C Basic Data Types	32-bit CPU		64-bit CPU	
	Size (bytes)	Range	Size (bytes)	Range
char	1	-128 to 127	1	-128 to 127
short	2	-32,768 to 32,767	2	-32,768 to 32,767
int	4	-2,147,483,648 to 2,147,483,647	4	-2,147,483,648 to 2,147,483,647
long	4	-2,147,483,648 to 2,147,483,647	8	-9,223,372,036,854,775,808-9,223,372,036,854,775,807
long long	8	9,223,372,036,854,775,808-9,223,372,036,854,775,807	8	9,223,372,036,854,775,808-9,223,372,036,854,775,807
float	4	3.4E +/- 38	4	3.4E +/- 38
double	8	1.7E +/- 308	8	1.7E +/- 308

Constants in C

- Constants refer to fixed values that the program may not alter during its execution.
- These fixed values are also called **literals**.
- Constants can be of any of the basic data types like *an integer constant, a floating constant, a character constant, or a string literal*.

Constants in C

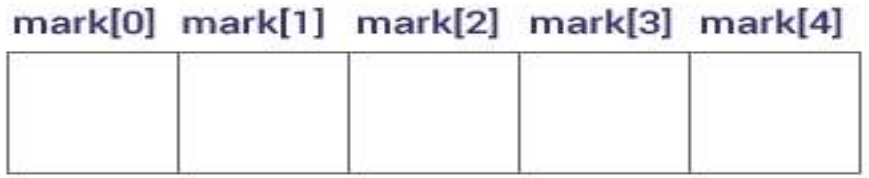


Introduction to Arrays

- An array is a collection of data that holds fixed number of values of same type.
- Syntax:

  data_type array_name[array_size]; ← Declaration

  float mark[5];



- The size and type of arrays cannot be changed after its declaration

Introduction to Arrays

- **Initialization**

  int mark[5] = {19, 10, 8, 17, 9}; ←Initialization

  int mark[] = {19, 10, 8, 17, 9};



Introduction to Arrays

- **Program to initialize an integer array with five values and prints the array.**

```
#include <stdio.h>
#include <conio.h>
int main()
{
int numbers[]={1,2,3,4,5};
int i;
clrscr();
printf("Array elements are\n");
for(i=0;i<=4;i++)
printf("%d\n",numbers[i]);
getch();
return 0;
}
```

Output

```
Array elements are
1
2
3
4
5
```

Introduction to Arrays

```
#include <stdio.h>
int main()
{
int marks[10], i, n, sum = 0, average;
printf("Enter n: ");
scanf("%d", &n);
for(i=0; i<n; ++i)
{
printf("Enter number%d: ",i+1);
scanf("%d", &marks[i]);
sum += marks[i];   //sum=sum+marks[i]
}
average = sum/n;
printf("Average = %d", average);
return 0;
}
```

Output

```
Enter n: 5
Enter number1: 45
Enter number2: 35
Enter number3: 38
Enter number4: 31
Enter number5: 49
Average = 39
```

2 Dimensional Arrays

- Two-dimensional array are those type of array, which has finite number of rows and finite number of columns.
- The declaration form of 2-dimensional array is

  Data_type Array_name [row size][column size];
- Example:

  int a[3][3];

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9

Example

```
#include<stdio.h>
#include<conio.h>
int main()
{
int matrix [3][3],i,j,r,c;
clrscr();
printf("Enter the order of matrix\n");
scanf("%d%d",&r,&c);
printf("Enter elements of %d * %d matrix \n",r,c);
for(i=0;i<r;i++)
for(j=0;j<c;j++)
scanf("%d",&matrix[i][j]);
printf("Given matrix:\n");
for(i=0;i<r;i++)
{
for(j=0;j<c;j++)
printf("%d\t",matrix[i][j]);
printf("\n");
}
return 0;
}
```

//Output

```
Enter the order of matrix
2
2
Enter elements of 2*2 matrix
1
2
3
4
Given matrix :
1   2
3   4
```

2 Dimensional Arrays

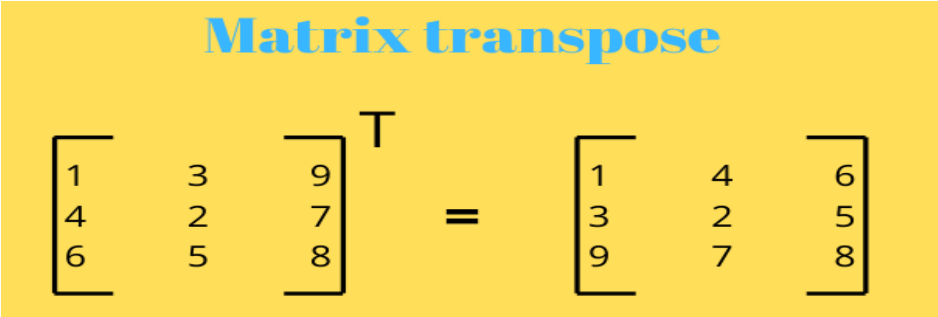
- Example: Matrix Operations
  - Addition
  - Subtraction
  - Scaling
  - Determinant
  - Transpose
  - Multiplication

2 D Arrays – Example Program
Matrix Addition

```
#include <stdio.h>
#include <conio.h>
void main()
{
int a[2][3],b[2][3],c[2][3],i,j;
clrscr();
printf("\nENTER VALUES FOR MATRIX A:\n");
for(i=0;i<2;i++)
     for(j=0;j<3;j++)
          scanf("%d",&a[i][j]);
printf("\nENTER VALUES FOR MATRIX B:\n");
for(i=0;i<2;i++)
     for(j=0;j<3;j++)
          scanf("%d",&b[i][j]);
for(i=0;i<2;i++)
     for(j=0;j<3;j++)
          c[i][j]=a[i][j]+b[i][j];
printf("\nTHE VALUES OF MATRIX C ARE:\n");
for(i=0;i<2;i++)
{
      for(j=0;j<3;j++)
           printf("%5d",c[i][j]);
           printf("\n");
}
getch();
}
```

2 D Arrays – Example Program Transpose of a matrix in C

- To obtain Transpose of a matrix, we interchange rows and columns of the matrix.



- For example, consider the following 3 X 2 matrix:
  1 2
  3 4
  5 6
  Transpose of the matrix:
  1 3 5
  2 4 6

2 D Arrays – Example Program
Transpose of a matrix in C

```
#include <stdio.h>
int main()
{
int m, n, c, d, matrix[10][10], transpose[10][10];
printf("Enter the number of rows and columns of a
matrix\n");
scanf("%d%d", &m, &n);


printf("Enter elements of the matrix\n");
for (c = 0; c < m; c++){
    for (d = 0; d < n; d++){
      scanf("%d", &matrix[c][d]);}}


for (c = 0; c < m; c++){
    for (d = 0; d < n; d++){
      transpose[d][c] = matrix[c][d];}}

printf("Transpose of the matrix:\n");

  for (c = 0; c < n; c++) {
    for (d = 0; d < m; d++)
      printf("%d\t", transpose[c][d]);
    printf("\n");
  }

return 0;
}
```

```
Enter the number of rows and columns of matrix
2
3
Enter the elements of matrix
1 2 3
4 5 6
Transpose of entered matrix :-
1       4
2       5
3       6
```

2 D Arrays – Example Program Determinant of a matrix in C

```
#include<stdio.h>
int main(){
int a[3][3], i, j;
long determinant;
printf("Enter the 9 elements of matrix: );
for(i = 0 ;i < 3;i++)
    for(j = 0;j < 3;j++)
        scanf("%d", &a[i][j]);

printf("\nThe matrix is\n");
for(i = 0;i < 3; i++){
    printf("\n");
    for(j = 0;j < 3; j++)
        printf("%d\t", a[i][j]);
}
```

```
determinant = a[0][0] * ((a[1][1]*a[2][2]) -
(a[2][1]*a[1][2])) -a[0][1] * (a[1][0]
* a[2][2] - a[2][0] * a[1][2]) + a[0][2] * (a[1][0] * a[2][1] -
a[2][0] * a[1][1]);
printf("\nDeterminant of 3X3 matrix: %ld",
determinant);
return 0;
}
```

```
Enter the 9 elements of matrix: 1 2 3 4 5 1 2 3 4
The matrix is

1       2       3
4       5       1
2       3       4
Determinant of 3X3 matrix: -5
```

2 D Arrays – Example Program
Scalar Matrix Multiplication

```
#include<stdio.h>
int main()
{
int i, j, row, col, Multi[10][10], Number;
printf("\n Please Enter Number of rows and columns\n");
scanf("%d %d", &i, &j);
printf("\n Please Enter the Matrix Elements \n");
for(row = 0; row < i; row++)
   for(col = 0;col < j; col++)
      scanf("%d", Multi[rows][columns]);
printf("\n Please Enter the Multiplication value: ");
scanf("%d", &Number);
for(row = 0; rows< i; row++)
  for(col = 0; col < j; col++)
        Multi[row][col] = Number *
              Multi[row][col];
printf("\n The Result of a Scalar Matrix Multiplication
is : \n");
for(row = 0; row < i; row++)
  for(col = 0; col < j; col++)
      printf("%d \t ", Multi[row][col]);
   printf("\n");
return 0; }
```

```
 Please Enter Number of rows and columns  : 3 3

 Please Enter the Matrix Elements
10 20 30
40 50 60
70 80 90

 Please Enter the Multiplication Value  :  3

 The Result of a Scalar Matrix Multiplication is :
30        60        90
120       150       180
210       240       270
```

2 D Arrays – Example Program
Matrix Multiplication

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad B = \begin{pmatrix} 5 & 6 & 7 \\ 8 & 9 & 10 \end{pmatrix}$$

Multiplication of two matrixes:

$$A * B = \begin{pmatrix} 1*5 + 2*8 & 1*6 + 2*9 & 1*7 + 2*10 \\ 3*5 + 4*8 & 3*6 + 4*9 & 3*7 + 4*10 \end{pmatrix}$$

$$A * B = \begin{pmatrix} 21 & 24 & 27 \\ 47 & 54 & 61 \end{pmatrix}$$

Javatpoint

2 D Arrays – Example Program Matrix Multiplication

```
#include<stdio.h>
#include<stdlib.h>
int main(){
int a[10][10],b[10][10],mul[10][10],r,c,i,j,k;
printf("enter the number of row=");
scanf("%d",&r);
printf("enter the number of column=");
scanf("%d",&c);
printf("enter the first matrix element=\n");
for(i=0;i<r;i++)  {
for(j=0;j<c;j++)   {
   scanf("%d",&a[i][j]);   }}
printf("enter the second matrix element=\n");
for(i=0;i<r;i++)  {
for(j=0;j<c;j++)   {
scanf("%d",&b[i][j]);  } }
printf("multiply of the matrix=\n");
for(i=0;i<r;i++)   {
for(j=0;j<c;j++)  {
mul[i][j]=0;
for(k=0;k<c;k++)   {
mul[i][j]+=a[i][k]*b[k][j];
} } }
//for printing result
for(i=0;i<r;i++)  {
for(j=0;j<c;j++)  {
printf("%d\t",mul[i][j]);
}
printf("\n");
}
return 0;

}
```

```
enter the number of row=3
enter the number of column=3
enter the first matrix element=
1 1 1
2 2 2
3 3 3
enter the second matrix element=
1 1 1
2 2 2
3 3 3
multiply of the matrix=
6 6 6
12 12 12
18 18 18
```

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

POINTER

Definition and Features of Pointer

➢ **Definition:**

- Pointer is a variable that stores the address of another variable.

➢ **Features**

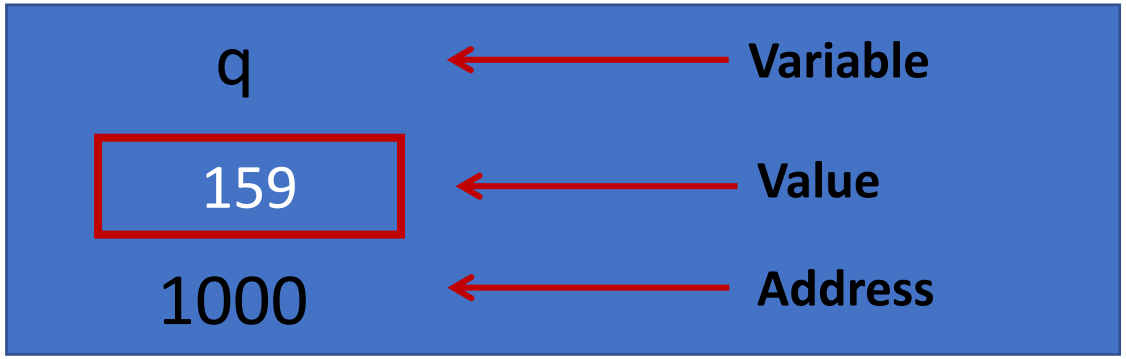
- Pointer saves the memory space.
- Execution time of pointer is faster because of direct access to the memory location.
- With the help of pointers, the memory is accessed efficiently, i.e., memory is allocated and deallocated dynamically.
- Pointers are used with data structures.

Pointer declaration, initialization and accessing

- Consider the following statement

int q = 159;

In memory, the variable can be represented as follows –



Pointer declaration, initialization and accessing(Cont..)

- **Declaring a pointer**

  It means ‘p’ is a pointer variable which holds the address of another integer variable, as mentioned in the statement below –

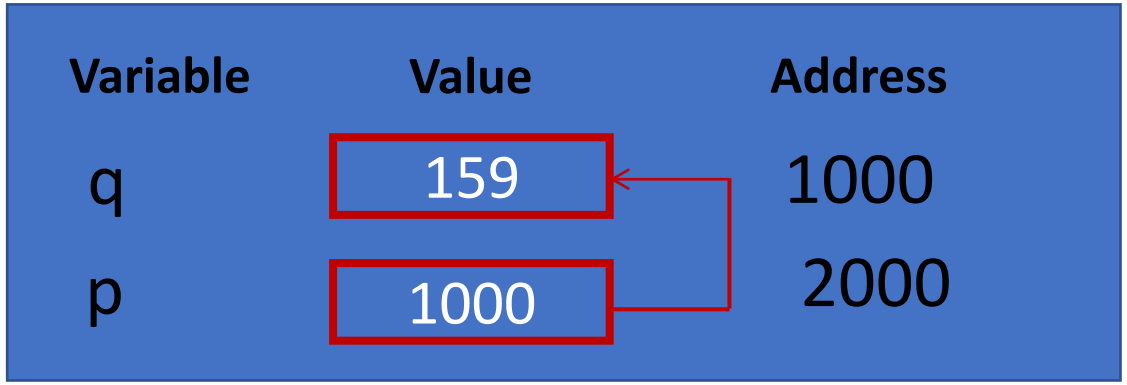
  ```
  int *p;
  ```

- **Initialization of a pointer**

  Address operator (&) is used to initialise a pointer variable.

  For example –

  ```
  int q = 159;
  int *p;
  p= &q;
  ```



Pointer declaration, initialization and accessing(Cont..)

- **Accessing a variable through its pointer**

To access the value of a variable, indirection operator (*) is used.

For example –

int q=159, *p,n;

p=&q;

n=*p

Here, '*' can be treated as value at address.

p = &q;

n = *p; n =q

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

1D INITIALIZATION & ACCESSING USING POINTERS

Pointers and one-dimensional arrays

- The compiler allocates Continuous memory locations for all the elements of the array.
- The base address = first element address (index 0) of the array.
- Syntax: data_type (var_name)[size_of_array]
  - For Example – int a [5] = {10, 20,30,40,50};
- **Elements**
- The five elements are stored as follows –

```
//Declaration
int *p;
 int my_arr[] = {11, 22, 33, 44, 55};
p = my_arr;
```

Elements	a[0]	a[1]	a[2]	a[3]	a[4]
Values	10	20	30	40	50
Address	1000	1004	1008	1012	1016

Base Address
a=&a[0]=1000

Pointers and one-dimensional arrays (Cont)

- If ‘p’ is declared as integer pointer, then, an array ‘a’ can be pointed by the following assignment –

  p = a; (or) p = &a[0];

- Every value of 'a' is accessed by using p++ to move from one element to another element. When a pointer is incremented, its value is increased by the size of the data type that it points to. This length is called the "scale factor".
- The relationship between ‘p’ and 'a' is explained below

  P = &a[0] = 1000

  P+1 = &a[1] = 1004

  P+2 = &a[2] = 1008

  P+3 = &a[3] = 1012

  P+4 = &a[4] = 1016

Pointers and one-dimensional arrays (Cont..)

- Address of an element is calculated using its index and the scale factor of the data type. An example to explain this is given herewith.
- Address of a[3] = base address + (3* scale factor of int)
  = 1000 + (3*4)
  = 1000 +12
  = 1012
- Pointers can be used to access array elements instead of using array indexing.
- *(p+3) gives the value of a[3].
- a[i] = *(p+i)

Example Program

- **Following is the C program for pointers and one-dimensional arrays –**

```
#include<stdio.h> main ( )
{
int a[5];
int *p,i;
printf ("Enter 5 lements");
for (i=0; i<5; i++)
scanf ("%d", &a[i]);
 p = &a[0];
printf ("Elements of the array are");
for (i=0; i<5; i++)
printf("%d", *(p+i));
}
```

Output

When the above program is executed, it produces the following result –

Enter 5 elements : 10 20 30 40 50

Elements of the array are : 10 20 30 40 50

Example Program : 1D array using Pointer

```
#include<stdio.h>
 int main()
 {
int array[5];
 int i,sum=0;
 int *ptr;
printf("\nEnter array elements (5 integer values):");
for(i=0;i<5;i++)
scanf("%d",&array[i]);
/* array is equal to base address * array = &array[0] */
ptr = array;
for(i=0;i<5;i++)
 {
sum = sum + *ptr; //*ptr refers to the value at address
ptr++;
 }
printf("\nThe sum is: %d",sum);
 }
```

Output

Enter array elements (5 integer values): 1 2 3 4 5

The sum is: 15

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

2D INITIALIZATION & ACCESSING USING POINTERS

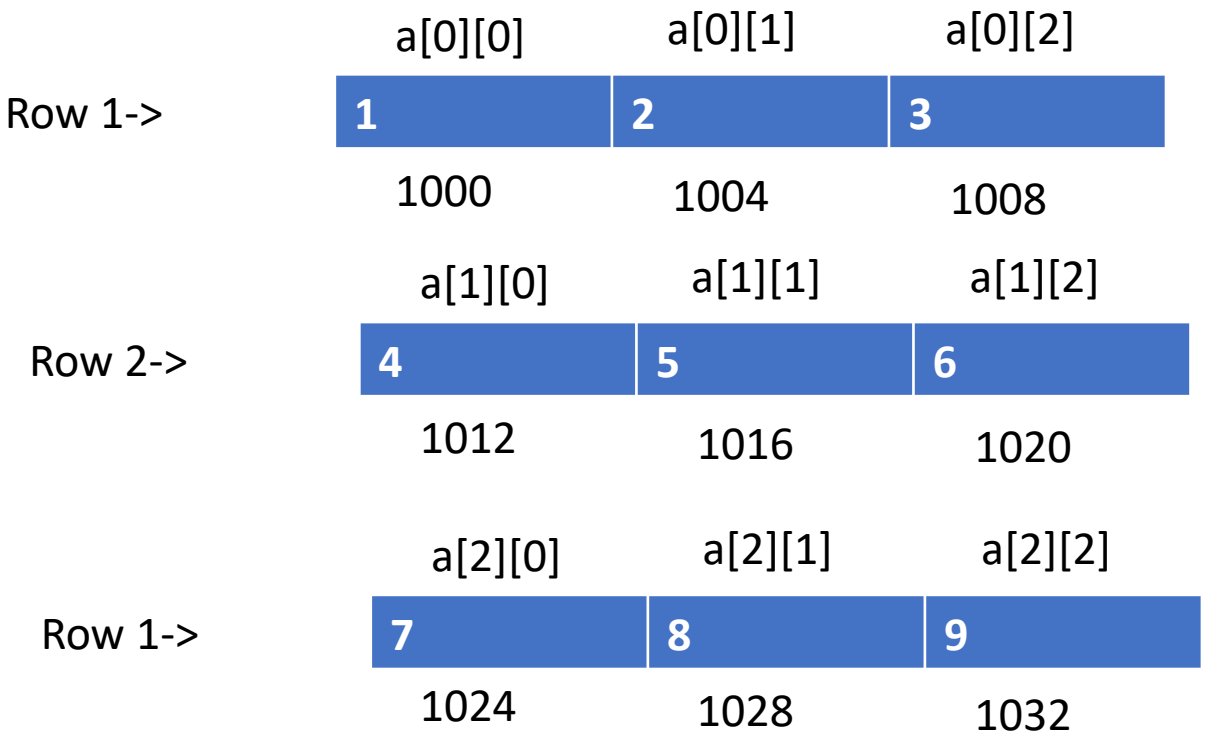
Pointers and two-dimensional arrays

- Memory allocation for a two-dimensional array is as follows –

int a[3] [3] = {1,2,3,4,5,6,7,8,9};

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

Base Address=1000=&a[0][0]



Pointers and two-dimensional arrays

- Assigning Base Address to pointer

```
int *p
p=&a[0][0] (or)
p=a
```

Pointer is used to access the elements of 2-dimensional array as follows

a[i][j]=*(p+i*columnsize+j)

a[1] [2] = *(1000 + 1*3+2)

= *(1000 + 3+2)

= *(1000 + 5*4) // 4 is Scale factor

= * (1000+20)

= *(1020)

a[1] [2] = 6

Example Program

```
#include<stdio.h> main ( )
{    int a[3] [3], i,j;
 int *p;
clrscr ( );
 printf ("Enter elements of 2D array");
  for (i=0; i<3; i++)
{
for (j=0; j<3; j++)
{
scanf ("%d", &a[i] [j]);
 }
 }
 p = &a[0] [0];
```

```
printf ("elements of 2d array are");
for (i=0; i<3; i++)
{
 for (j=0; j<3; j++)
{
printf ("%d \t", *(p+i*3+j));
    }
 printf ("\n");
}
getch ( );
}
```

Output

When the above program is executed, it produces the following result –

enter elements of 2D array 1 2 3 4 5 6 7 8 9

Elements of 2D array are

1 2 3

4 5 6

7 8 9

Review Questions

1. Let's consider 60 students took this course. Each student is supposed to give a rating 1, 2, 3 4 or 5; one being bad and 5 being good. Find the ratings are spread count the number of times each rating was given.
2. Write a Program in C for the following **Array** operations
    a. Creating an Array of **N** Integer Elements
    b. Display of Array Elements with Suitable Headings
    c. Inserting an Element (**ELEM**) at a given valid Position (**POS**)
    d. Deleting an Element at a given valid Position(**POS**)
    e. Exit.

Support the program with functions for each of the above operations.

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

DECLARING STRUCTURE AND ACCESSING

Definition of Structures

- Structures are user defined data types
- It is a collection of heterogeneous data
- It can have integer, float, double or character data in it
- A structure is a collection of variables of different data types grouped under a single name.
- struct<<structname>>

```
        {
            members;
        }element;
```

We can access element.members;

Array Vs Structure

Difference Between Array and Structure

Array	Structure
Array is collection of homogeneous data.	Structure is the collection of heterogeneous data.
Array data are access using index.	Structure elements are access using . operator.
Array allocates static memory.	Structures allocate dynamic memory.
Array element access takes less time than structures.	Structure elements takes more time than Array.



Declaration of Structures

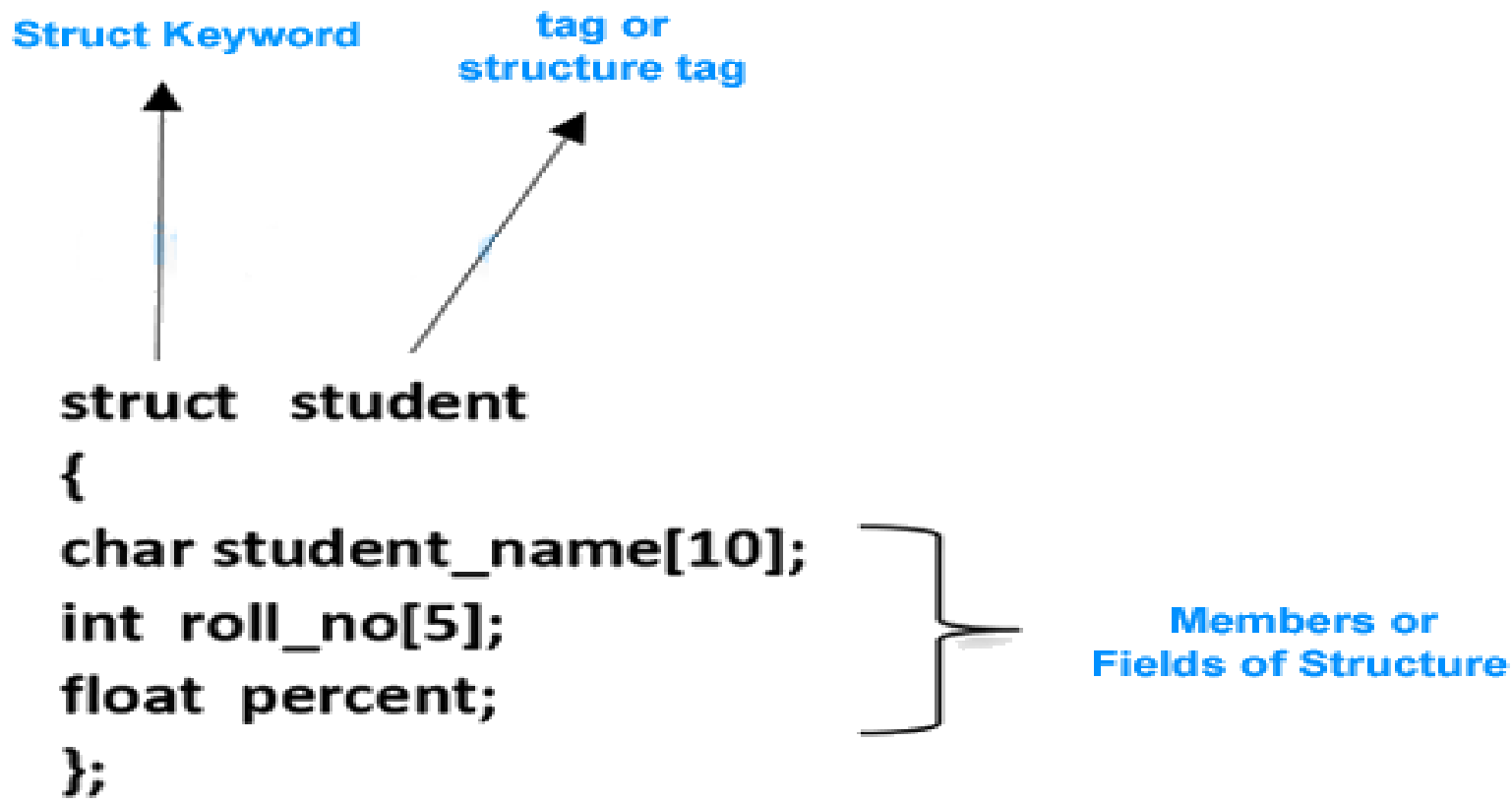
Syntax:

```
struct  struct_Name
{
<data-type> member_name;
<data-type> member_name;
…
} list of variables;
```

Example: Structure to hold student details

```
struct Student
{
        int regNo; char
        name[25];
        int english,science,maths;
} stud;
```

Example


Struct Keyword
tag or
structure tag
struct student
{
char student_name[10];
int roll_no[5];
float percent;
};
Members or
Fields of Structure

Example

```
struct emp
{
int id;
char name[36];
int sal;
};
sizeof(struct emp) // --> 40 byte
(2byte+36byte+2byte)
```

Declaring a structure variable

```
struct <structure name>
 {
        Datatype1 member1;
        Datatype2 member2;
        Datatype3 member3;
        ..........
} variable1,variable2, …, variablen;
            (OR)
struct  <structure name>
{
…
…
} ;
struct <structure name>  variable1,variable2, …, variablen;
```

Declaration of Structures

Syntax:

```
struct struct_Name
{
        <data-type> member_name;
        <data-type> member_name;
        …
};
struct struct_Name struct_var;
```

```
struct Student
{
        int regNo;
        char name[25];
        int english,science, maths;
};
struct Student stud;
```

Initialization of Structures

```
// Static Initialization of stud
stud.regNo=1425;
strcpy(stud.name,"Banu");
stud.English=78;
stud.science=89;
stud.maths=56;
// Dynamic Initialization of stud
scanf("%d %s %d %d %d",&stud.regNo, stud.name, &stud.English,&stud.science, &stud.maths);
```

Accessing and Updating Structures

```
struct Student
{
        int regNo;char name[25];
        int english,science,maths;
};
struct Student stud;
scanf("%d %s%d%d%d",&stud.regNo,stud.name,&stud.english,&stud.science,&stud.maths);

printf('Register number =%d \n",stud.Regno);          //Accessing member Regno
printf('Name %s",stud.name);                          //Accessing member name
stud.english=stud.english+10;                         //Updating member english
stud.science++;                                       //Updating member science
```

Example Program

```
#include<stdio.h>
#include<conio.h>
struct emp
{
int id;
char name[36];
float sal;
};
void main()
{
struct emp e;
clrscr();
printf("Enter employee Id, Name, Salary: ");
scanf("%d",&e.id); scanf("%s",&e.name);
scanf("%f",&e.sal);
printf("Id: %d",e.id); printf("\nName:
%s",e.name); printf("\nSalary: %f",e.sal);
getch();
}
```

Example Program

```
#include <stdio.h>
#include <string.h>
struct Books
{
char title[50];
char author[50];
char subject[100];
int book_id;
};

int main( )
{
struct Books Book1; /* Declare Book1 of type Book */
struct Books Book2; /* Declare Book2 of type Book */

/* book 1 specification */ strcpy( Book1.title, "C
Programming"); strcpy( Book1.author, "Nuha
Ali"); strcpy( Book1.subject, "C Programming
Tutorial"); Book1.book_id = 6495407
```

Example Program

```
/* book 2 specification */ strcpy( Book2.title,
"Telecom Billing"); strcpy( Book2.author, "Zara
Ali"); strcpy( Book2.subject, "Telecom Billing
Tutorial"); Book2.book_id = 6495700;

/* print Book1 info */
printf( "Book 1 title : %s\n", Book1.title);
printf( "Book 1 author : %s\n", Book1.author);
printf( "Book 1 subject : %s\n", Book1.subject);
printf( "Book 1 book_id : %d\n", Book1.book_id);
 /* print Book2 info */
printf( "Book 2 title : %s\n", Book2.title);
printf( "Book 2 author : %s\n", Book2.author);
printf( "Book 2 subject : %s\n", Book2.subject);
printf( "Book 2 book_id : %d\n", Book2.book_id);
return 0;
}
```

OUTPUT

```
Book 1 title : C Programming
Book 1 author : Nuha Ali
Book 1 subject : C Programming Tutorial
Book 1 book_id : 6495407
Book 2 title : Telecom Billing
Book 2 author : Zara Ali
Book 2 subject : Telecom Billing Tutorial
Book 2 book_id : 6495700
```

Example Program

```
#include<stdio.h>
struct student
{
    char name[20];
    int id;
    float marks;
};
void main()
{
    struct student s1,s2,s3;
    printf("Enter the name, id, and marks of student 1 ");
    scanf("%s %d %f",s1.name,&s1.id,&s1.marks);
    printf("Enter the name, id, and marks of student 2 ");
    scanf("%s %d %f",s2.name,&s2.id,&s2.marks);
    printf("Enter the name, id, and marks of student 3 ");
    scanf("%s %d %f",s3.name,&s3.id,&s3.marks);
    printf("Printing the details....\n");
    printf("%s %d %f\n",s1.name,s1.id,s1.marks);
    printf("%s %d %f\n",s2.name,s2.id,s2.marks);
    printf("%s %d %f\n",s3.name,s3.id,s3.marks);
}
```

Output

```
Enter the name, id, and marks of student 1
James
90
90
Enter the name, id, and marks of student 2
Adoms
90
90
Enter the name, id, and marks of student 3
Nick
90
90
Printing the details....
James 90 90.000000
Adoms 90 90.000000
Nick 90 90.000000
```

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

DECLARING ARRAY OF STRUCTURES AND ACCESSING

Array of Structures

- Structure – for storing information of one particular object
- Array of structure - used to store 100 such objects
- Example

```
struct Bookinfo
{
    char bname[20];
    int pages;
    int price;
}Book[100];
```

Array of Structures

```
struct Student
        {
                int regNo;
                char
                name[25];
                int english, science, maths;
        };
struct Student stud[10];
scanf("%d%s%d%d%d",&stud[1].regNo,stud[1].name,&stud[1].english,&stud[1].science,
 &stud[1].maths);
printf("Register number = %d\n",stud[1].Regno);          //Accessing member Regno
printf("Name %s",stud[1].name);                          //Accessing member name
stud.[1]english= stud[0].english +10;                    //Updating member english
stud[1].science++;                                       //Updating member science
```

Example Program

```
#include<stdio.h>
#include <string.h>
struct student{
int rollno;
char name[10];
};

int main()
{
int i;
struct student st[5];
printf("Enter Records of 5 students");
for(i=0;i<5;i++){
printf("\nEnter Rollno:");
scanf("%d",&st[i].rollno);
printf("\nEnter Name:");
scanf("%s",&st[i].name);
}
printf("\nStudent Information List:");
for(i=0;i<5;i++){
printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
}
  return 0;
}
```

OUTPUT

```
Enter Records of 5 students
Enter Rollno:1 Enter Name:Sonoo
Enter Rollno:2 Enter Name:Ratan
Enter Rollno:3 Enter Name:Vimal
Enter Rollno:4 Enter Name:James
Enter Rollno:5 Enter Name:Sarfraz
Student Information List:
Rollno:1, Name:Sonoo
Rollno:2, Name:Ratan
Rollno:3, Name:Vimal
Rollno:4, Name:James
Rollno:5, Name:Sarfraz
```

Array of Structures - Example

Program to store and display the records of 3 students.

```
#include <stdio.h>
#include <string.h>
struct student
{
int id;
char name[30];
float percentage;
};
int main()
{
int i;
struct student record[3];
// 1st student's record
record[0].id=1;
strcpy(record[0].name, "Raju");
record[0].percentage = 86.5;
// 2nd student's record
record[1].id=2;
strcpy(record[1].name, "Surendren");
record[1].percentage = 90.5;
// 3rd student's record
record[2].id=3;
strcpy(record[2].name, "Thiyagu");
record[2].percentage = 81.5;
for(i=0; i<3; i++)
{
printf(" Records of STUDENT : %d \n", i+1);
printf(" Id is: %d \n", record[i].id);
printf(" Name is: %s \n", record[i].name);
printf(" Percentage is: %f\n\n",record[i].percentage);
}
return 0;
```

Example: Program to store and display the details of 50 students

```
#include <stdio.h>
#include <string.h>
struct student
{
int id;
char name[30];
float percentage;
};
int main()
{
int i;
struct student record[50];
printf("Enter the record details\n");
for(i=0;i<50;i++)
{
  scanf( "%d%s%f", &record[i].id,  &record[i].name, &record[i].percentage);
}
#Display the details
for(i=0; i<50; i++)
{
printf(" Records of STUDENT : %d \n", i+1);
printf(" Id is: %d \n", record[i].id);
printf(" Name is: %s \n", record[i].name);
printf(" Percentage is: %f\n\n",record[i].percentage);
}
return 0;
```

Nested Structure

- When a structure contain another structure, it is called nested structure.
- Syntax

```
struct structure1
{
- - - - - - - - - - -
- - - - - - - - - - -
};
struct structure2
{
- - - - - - - - - - -
- - - - - - - - - - -
struct structure1 obj;
};
```

Nested Structures

```
struct Student
{
        int regNo;
        char name[25];
        struct marks
        {       int english,science,maths;    }
}Amar;
```

```
struct Student
{
        int regNo;
        char name[25];
        struct {int english,science,maths } marks;
}Amar;
```

```
struct Student
{
        int regNo;
        char name[25];
        struct m marks;
}Amar;
```

```
struct m
{
        int english;
        int science;
        int maths;
};
```

```
Amar.regNo;
Amar.marks.english;
Amar.marks.science;
Amar.marks.maths;
```

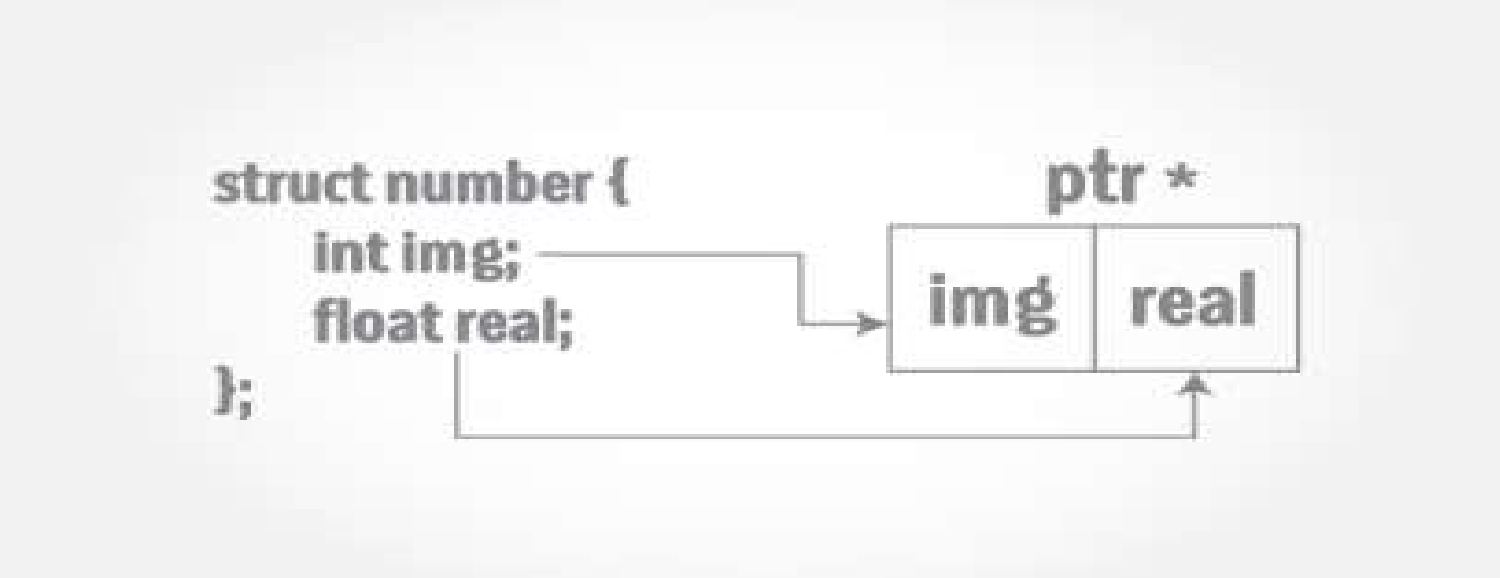
Nested Structure - Example

```
#include<stdio.h>
struct Address
{
char HouseNo[25];
char City[25];
char PinCode[25];
};
struct Employee
{
int Id;
char Name[25];
float Salary;
struct Address Add;
};
void main()
{
printf("\n\tEnter Employee Salary : ");
{
int i;
struct Employee E;
printf("\n\tEnter Employee Id : ");
scanf("%d",&E.Id);
printf("\n\tEnter Employee Name : ");
scanf("%s",&E.Name);
printf("\n\tEnter Employee Salary : ");
scanf("%f",&E.Salary);
printf("\n\tEnter Employee House No : ");
scanf("%s",&E.Add.HouseNo);
printf("\n\tEnter Employee City : ");
scanf("%s",&E.Add.City);
printf("\n\tEnter Employee House No : ");
scanf("%s",&E.Add.PinCode);
printf("\nDetails of Employees");
printf("\n\tEmployee Id : %d",E.Id);
printf("\n\tEmployee Name : %s",E.Name);
printf("\n\tEmployee Salary : %f",E.Salary);
printf("\n\tEmployee House No
:%s",E.Add.HouseNo);
printf("\n\tEmployee City : %s",E.Add.City);
printf("\n\tEmployee House No :
%s",E.Add.PinCode);
```

Pointers and Structures

- Structures can be created and accessed using pointers.

```
int a;
int *p;
p=&a;
```



Pointers and Structures

- A pointer variable of a structure can be created as follows:

```
struct name
{
member1;
member2;
.
.
};
int main()
{
struct name *ptr;
}
```

Accessing structure's member through pointer

Referencing pointer to another address to access memory

```
#include <stdio.h>
typedef struct person
{
int age;
float weight;
};
```

```
int main()
{
struct person *personPtr, person1;
personPtr = &person1; // Referencing pointer to memory address of person1
printf("Enter integer: ");
scanf("%d",&(*personPtr).age);
printf("Enter number: ");
scanf("%f",&(*personPtr).weight);
printf("Displaying: ");
printf("%d%f",(*personPtr).age,(*personPtr).weight);
return 0;
}
```

Dynamic memory allocation

- Dynamic memory allocation allows your program to obtain more memory space while running, or to release it if it's not required.
- library functions under "stdlib.h" for dynamic memory allocation

Function	Use of Function
malloc()	Allocates requested size of bytes and returns a pointer first byte of allocated space
calloc()	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
free()	deallocate the previously allocated space
realloc()	Change the size of previously allocated space

Dynamic Memory Allocation

- **malloc()**
  - The name malloc stands for "memory allocation".
  - The function malloc() reserves a block of memory of specified size and return a pointer of type void which can be casted into pointer of any form.
- **Syntax of malloc()**
  - ptr = (cast-type*) malloc(byte-size)

    ptr = (int*) malloc(100 * sizeof(int));

Dynamic Memory Allocation

- **calloc()**
  - The name calloc stands for "contiguous allocation".
  - The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory.
- **Syntax of calloc()**
  - ptr = (cast-type*)calloc(n, element-size);
  - This statement will allocate contiguous space in memory for an array of n elements.
- example:
  - ptr = (float*) calloc(25, sizeof(float));

Dynamic Memory Allocation

- **free()**
  - Dynamically allocated memory created with either calloc() or malloc() doesn't get
  - freed on its own. You must explicitly use free() to release the space.
- **syntax of free()**
  - free(ptr);

Dynamic Memory Allocation

Sum of Element in an array

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
int num, i, *ptr, sum = 0;
printf("Enter number of elements in the Array: ");
scanf("%d", &num);
ptr = (int*) malloc(num * sizeof(int));
if(ptr == NULL)
{
printf("Error! Unable to allocate memory.");
exit(0);
}
printf("Enter elements of array: ");
for(i = 0; i < num; ++i)
{
scanf("%d", ptr + i);
}

for(i = 0; i < num; ++i)
{
sum += *(ptr + i);
}
printf("Sum of Elements in the given array is : %d", sum);
free(ptr);
return 0;
}
```

Output:

```
Enter the elements in the array:4
Enter elements of array:7
20
3
5
Sum of Elements in the given array:35
```

Accessing structure's member through pointer using dynamic memory allocation

```
#include <stdio.h>
#include <stdlib.h>
struct person {
int age;
float weight;
char name[30];
};
int main()
{
struct person *ptr;
int i, num;
printf("Enter number of persons: ");
scanf("%d", &num);
ptr = (struct person*) malloc(num *
sizeof(struct person));
// Above statement allocates the memory
for n structures with pointer personPtr
pointing to base address */
for(i = 0; i < num; ++i)
{
    printf("Enter name, age and weight of
    the person respectively:\n");
    scanf("%s%d%f", &(ptr+i)->name,
    &(ptr+i)->age,
    &(ptr+i)->weight);
}
printf("Displaying Information:\n");
for(i = 0; i < num; ++i)
    printf("%s\t%d\t%.2f\n", (ptr+i)-
    >name, (ptr+i)->age, (ptr+i)->weight);
return 0;
}
```

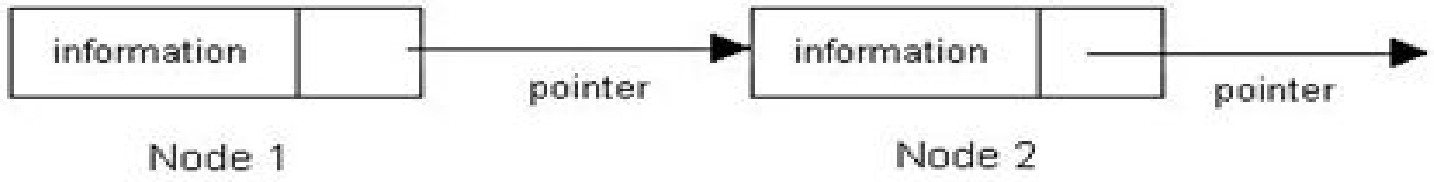
Self-referential structures

- A structure which contains a pointer to itself is known as self referential structures.

```
struct name
{
        member 1;
        member 2;
        . . .
        struct name *pointer;
};
```

Self-referential structures

- Example for Self referential structure is – Linked List
- Linked List is a collection of nodes which are not necessary to be in adjacent memory location.
- Each node – 2 fields (data field, address field)

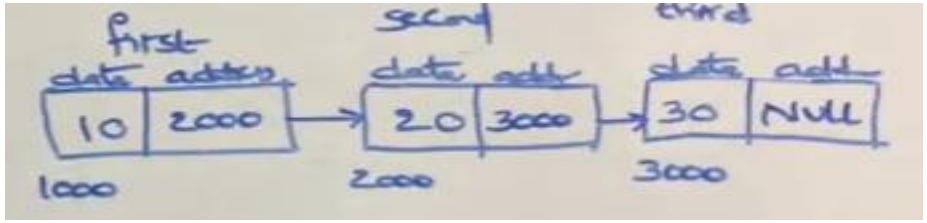


Self-referential structures

- Singly Linked List
- Circular Linked List
- Doubly Linked List
- Circular double linked list

Program implement Self-referential structures

```
#include<stdio.h>
main()
{
struct node
{
        int data;
        struct node *addr;
}*first,*second, *third;
printf("Enter the Ist node data);
scanf('%d", &first→data);
first→addr = second;
printf("Enter the 2nd node data);
scanf('%d", &second→data);
first→addr = third;
printf("Enter the 3rd node data);
scanf('%d", &third→data);
second→addr = second;
third→addr=NULL;
temp=first;
while(temp!= NULL)
{
        printf("%d", temp→data);
        temp = temp→addr;
}
}
```



Review Questions

1. Write a C program for student mark sheet using structure
2. Write a C program using structure to find students grades in a class
3. Write a C program for book details using structure
4. Calculating the area of a rectangle using structures in C language.

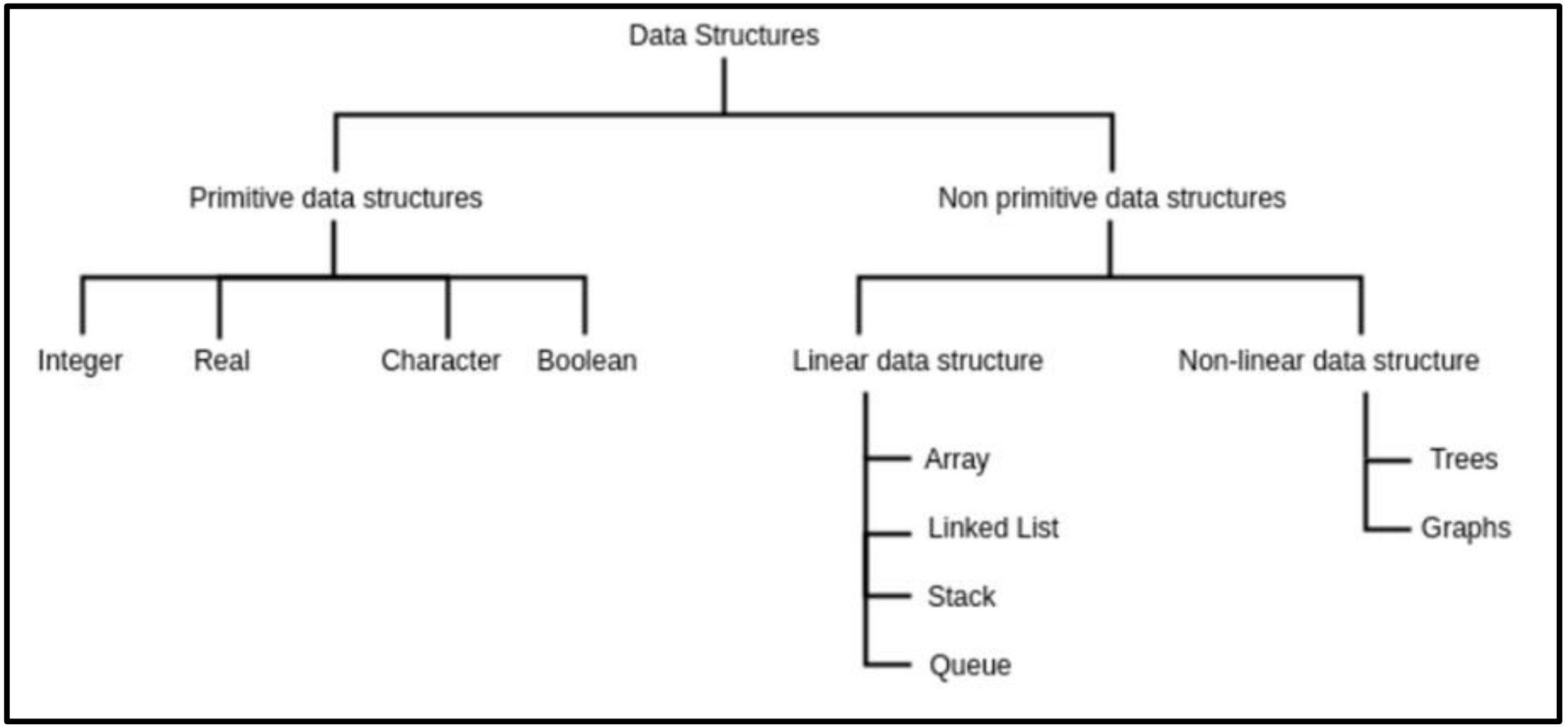
SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

DATA STUCTURES

Data Structures

- Data Structure can be defined as the **group of data elements** which provides an efficient way of **storing and organizing data** in the computer so that it can be used efficiently.
- Data Structures are the main part of many computer science algorithms as they enable the programmers to handle the data in an efficient way.
- It plays a vital role in enhancing the performance of a software or a program as the main function of the software is to store and retrieve the user's data as fast as possible.

Classification of Data Structures



Primitive & Non-Primitive of Data Structures

- **Primitive data structures** are the fundamental data types which are supported by a programming language.
- Some basic data types are integer, real(float), character, and Boolean.
- **Non-primitive data structures** are data structures which are created using primitive data structures.
- Examples of such data structures include linked lists, stacks, trees, and graphs.
- Non-primitive data structures can further be classified into two categories: **linear and non-linear data structures.**

Linear Data Structures

- If the elements of a data structure are **stored in a linear or sequential order**, then it is a linear data structure.
- **Examples include arrays, linked lists, stacks, and queues.**
- Linear data structures can be represented in memory in two different ways.
- One way is to have a linear relationship between **elements by means of sequential memory locations.**
- The other way is to have a linear relationship between **elements by means of links.**

Non Linear Data Structures

- If the elements of a data structure are **not stored in a sequential order**, then it is a non-linear data structure.
- The relationship of adjacency is **not maintained** between **elements of a non-linear data structure.**
- **Examples include trees and graphs.**

Review Questions

- A hospital maintains a patient file in which each record contains the following data:

  Name, Admission date, Social security number, Room, Bed number, Doctor

  a) Which items can serve as primary keys?
  b) Which pair of items can serve as a primary key?
  c) Which items can be group items?

- Which of the following data items may lead to variable length records when included as items in the record:

  (a) age, (b) sex, (c) name of spouse, (d) names of children, (e) education,

  (f) previous employers

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

DATA STUCTURE OPERATIONS

Data Structure Operations

- The data in the data structures are processed by certain operations.
  - Traversing
  - Searching
  - Inserting
  - Updating
  - Deleting
  - Sorting
  - Merging

Data Structure Operations (Cont..)

- **Traversing** - Visiting each record so that items in the records can be accessed.
- **Searching** - Finding the location of the record with a given key or value or finding all records which satisfy given conditions.
- **Inserting** - Adding a new record to the data structure.
- **Updating** – Change the content of some elements of the record.
- **Deleting** - Removing records from the data structure.
- **Sorting** - Arranging records in some logical or numerical order.
  - **(Eg: Alphabetic order)**
- **Merging** - Combing records from two different sorted files into a single file.

Example

- An organization contains a membership file in which each record contains the following data for a given member:

  Name, Address, Telephone Number, Age, Sex

  a. Suppose the organization wants to announce a meeting through a mailing. Then one would **traverse** the file to obtain Name and Address of each member.

  b. Suppose one wants to find the names of all members living in a particular area. Again **traverse and search** the file to obtain the data.

  c. Suppose one wants to obtain Address of a specific Person. Then one would **search** the file for the record containing Name.

  d. Suppose a member dies. Then one would **delete** his or her record from the file.

Example

e. Suppose a new person joins the organization. Then one would **insert** a record into the file

f. Suppose a member has shifted to a new residence his/her address and telephone number need to be changed. Given the name of the member, one would first need to **search** for the record in the file. Then perform the **update** operation, i.e. change some items in the record with the new data.

g. Suppose one wants to find the number of members whose age is 65 or above. Again one would **traverse** the file, counting such members.

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

DATA STRUCTURES AND ITS TYPES

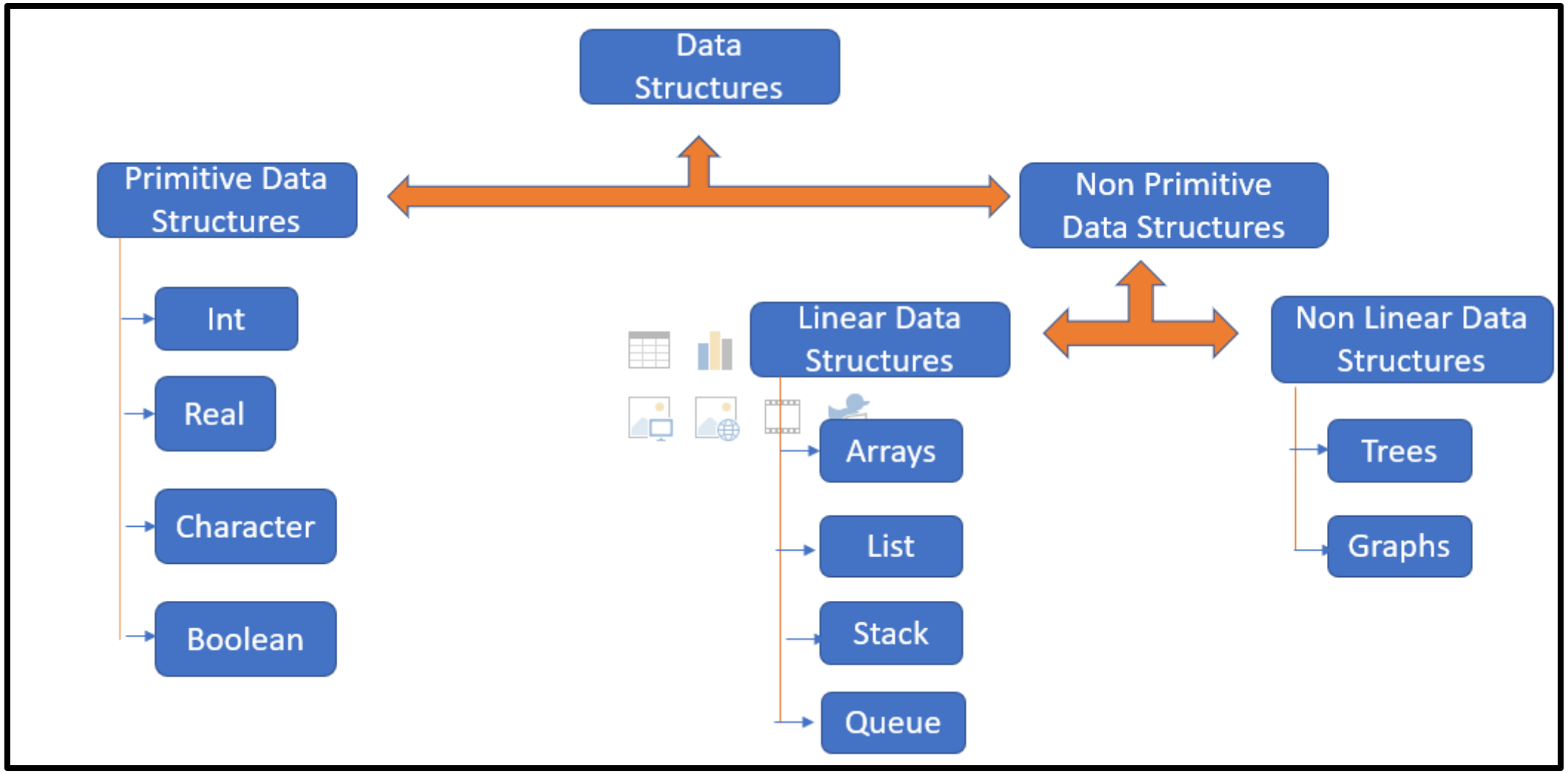
Data Structure

- A data structure is basically a group of data elements that are put together under one name
- Defines a particular way of storing and organizing data in a computer so that it can be used efficiently
- Data Structures are used in
  - Compiler design
  - Operating system
  - Statistical analysis package
  - DBMS
- The selection of an appropriate data structure provides the most efficient solution

Terms in Data Structure

- **Data:** Data can be defined as an elementary value or the collection of values, for example, student's name and its id are the data about the student.
- **Group Items:** Data items which have subordinate data items are called Group item, for example, name of a student can have first name and the last name.
- **Record:** Record can be defined as the collection of various data items, for example, if we talk about the student entity, then its name, address, course and marks can be grouped together to form the record for the student.
- **File:** A File is a collection of various records of one type of entity, for example, if there are 60 employees in the class, then there will be 20 records in the related file where each record contains the data about each employee.
- **Attribute and Entity:** An entity represents the class of certain objects. it contains various attributes. Each attribute represents the particular property of that entity.
- **Field:** Field is a single elementary unit of information representing the attribute of an entity.

Types in Data Structure



Types of Data Structure

Primitive Data Structures:

Primitive data structures are the fundamental data types which are supported by a programming language. Some basic data types are

- Integer
- Real
- Character
- Boolean

Non-Primitive Data Structures:

Non-primitive data structures are those data structures which are created using primitive data structures. Examples of such data structures include

- linked lists
- stacks
- trees
- graphs

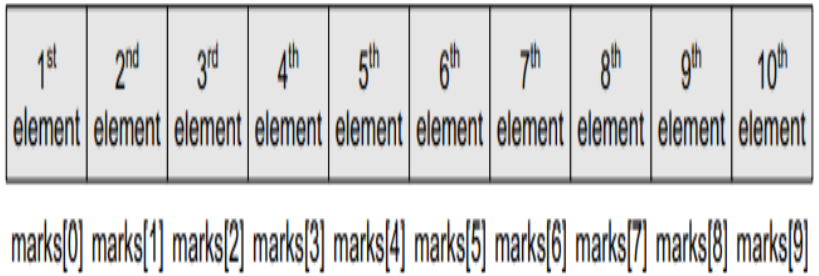
SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

LINEAR AND NON-LINEAR DATA STRUCTURES

Types of Data Structure – Non Primitive Linear

Arrays

- An array is a collection of similar type of data items and each data item is called an element of the array.



Linked List

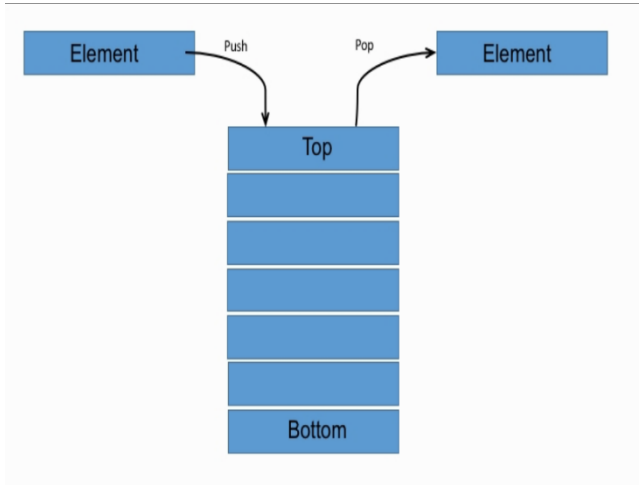
- Dynamic data structure in which elements (called nodes) form a sequential list



Types of Data Structure – Non Primitive Linear

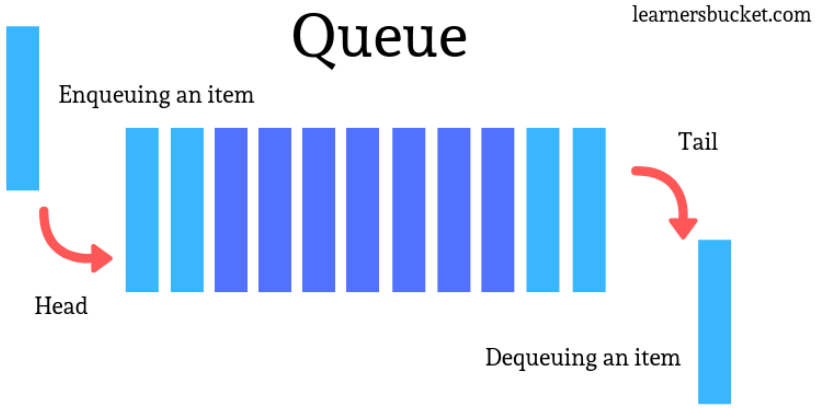
Stack

- Linear data structure in which insertion and deletion of elements are done at only one end, which is known as the top of the stack



Queue

- Linear data structure in which the element that is inserted first is the first one to be taken out



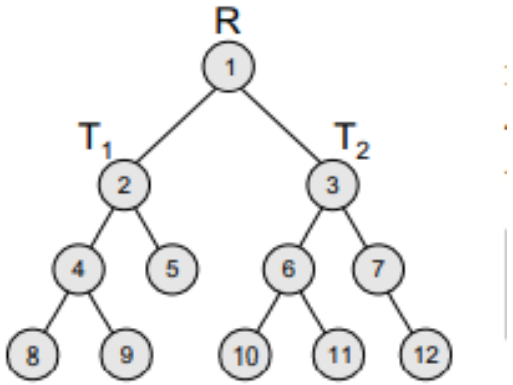
Types of Data Structure – Non Primitive - Non Linear

- The data structure where data items are not organized sequentially is called non linear data structure
- Types
  - Trees
  - Graphs

Types of Data Structure – Non Primitive - Non Linear

TREES

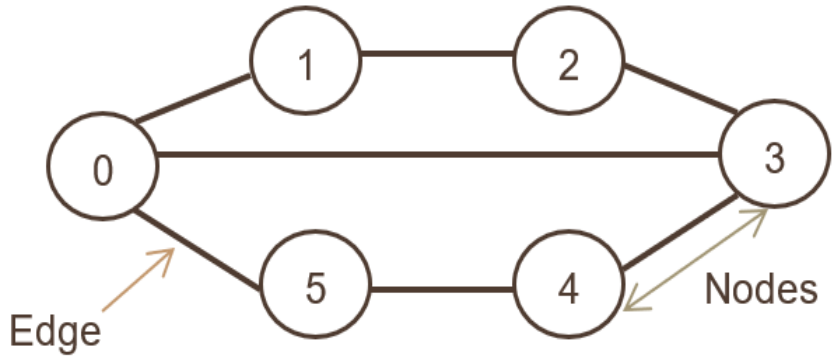
- A tree is a non-linear data structure which consists of a collection of nodes arranged in a hierarchical order.
- One of the nodes is designated as the root node, and the remaining nodes can be partitioned into disjoint sets such that each set is a sub-tree of the root.



Types of Data Structure – Non Primitive - Non Linear

GRAPHS

- A graph is a non-linear data structure which is a collection of vertices (also called nodes) and edges that connect these vertices.
- A graph is often viewed as a generalization of the tree structure.
- Every node in the graph can be connected with another node in the graph.
- When two nodes are connected via an edge, the two nodes are known as neighbours.



Review Questions

1. Compare a linked list with an array.
2. Is Array static structure or dynamic structure (TRUE / FALSE)
3. The data structure used in hierarchical data model is _____
4. In a stack, insertion is done at _____
5. Which Data Structure follows the LIFO fashion of working?
6. The position in a queue from which an element is deleted is called as _____

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

ABSTRACT DATA TYPE

Abstract Data Types

- An Abstract Data type (ADT) is a type for objects whose behavior is defined by a **set of value and a set of operations**.
- ADT refers to a set of data values and associated operations that are specified accurately, independent of any particular implementation.
- The ADT consists of a set of definitions that allow us to use the functions while hiding the implementation.
- Abstract in the context of data structures means everything except the detailed specifications or implementation.
- Data type of a variable is the set of values that the variable can take.

ADT = Type + Function Names + Behaviour of each function

- **Examples: Stacks, Queues, Linked List**

ADT Operations

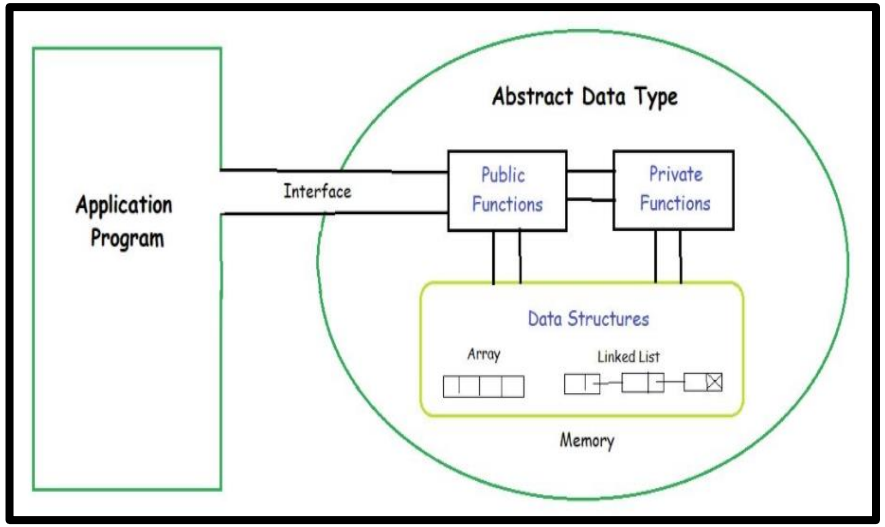
- While modeling the problems the necessary details are separated out from the unnecessary details. This process of modeling the problem is called abstraction.
- The model defines an abstract view to the problem. It focuses only on problem related stuff and that you try to define properties of the problem.
- These properties include
  - The data which are affected.
  - The operations which are identified.

ADT Operations (Cont..)

- Abstract data type operations are
  - **Create -** Create the data structure.
  - **Display -** Displaying all the elements stored in the data structure.
  - **Insert -** Elements can be inserted at any desired position.
  - **Delete -** Desired element can be deleted from the data structure.
  - **Modify -** Any desired element can be deleted/modified from the data structure.

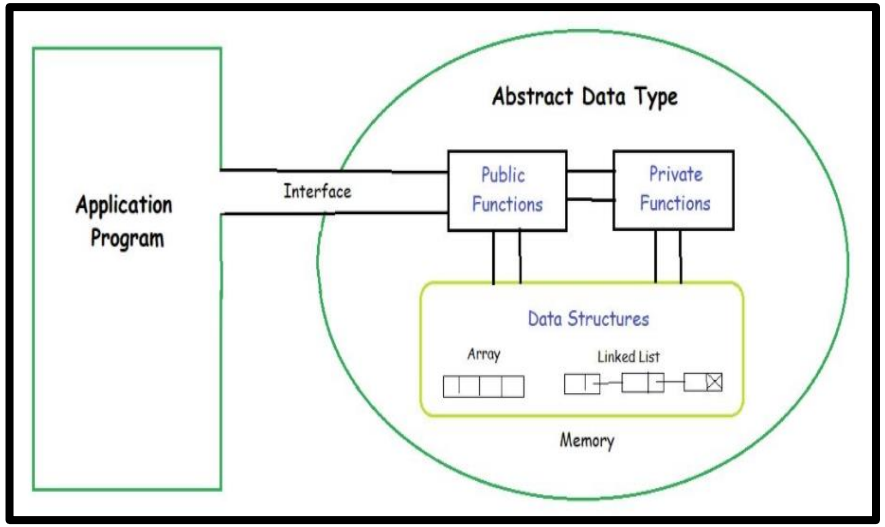
Abstract Data Types Model

- The ADT model has two different **parts functions (public and private) and data structures.**
- Both are contained within the ADT model itself, and do not come within the scope of the application program.
- Data structures are available to all of the ADT's functions as required, and a **function may call any other function** to accomplish its task.
- Data structures and functions are within the scope of each other.



Abstract Data Types Model (Cont..)

- Data are entered, accessed, modified and deleted through the external application programming interface.
- For each ADT operation, there is an algorithm that performs its specific task.
- The operation name and parameters are available to the application, and they provide only an interface to the application.
- When a list is controlled entirely by the program, it is implemented using simple structure.



Review Questions

- _____ are used to manipulate the data contained in various data structures.
- In _____, the elements of a data structure are stored sequentially.
- _____ of a variable specifies the set of values that the variable can take.
- Abstract means _____.
- If the elements of a data structure are stored sequentially, then it is a _____.

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

MATHEMATICAL NOTATIONS

1. Floor and Ceiling Functions

- If x is a real number, then it means that x lies between two integers which are called the floor and ceiling of x. i.e.
  - ⌊x⌋ **is called the floor of x. it is the greatest integer that is not greater than x.**
  - ⌈ x ⌉ **is called the ceiling of x. it is the smallest integer that is not less than x.**
- If x is itself an integer, then ⌊x⌋ = ⌈ x ⌉, otherwise ⌊x⌋ + 1 = ⌈ x ⌉
- E.g.
  - ⌊3.14⌋=3, ⌊-8.5⌋=-9, ⌊7⌋=7
  - ⌈ 3.14 ⌉=3, ⌈ -8.5 ⌉=-8, ⌈ 7 ⌉=7

2. Remainder Function (Modular Arithmetic)

- If k is any integer and M is a positive integer then,
  - **K (mod M)**
- Gives the integer remainder when k is divided by M
- E. g.
  - 25 (mod 7) = 4
  - 25 (mod 5) = 0

3. Integer and Absolute Value Functions

- If x is a real number, then integer function INT(x) will convert x into integer and the fractional part is removed.
  E.g.
  - **INT (3.14) = 3**
  - **INT (-8.5) = -8**

- The absolute function ABS(x) or | x | gives the absolute value of x i.e. it gives the positive value of x even if x is negative.
  E.g
  - **ABS(-15) = 15** **or** **AB | -15 | = 15**
  - **ABS(7) = 7** **or** **AB | 7 | = 7**
  - **ABS(-3.33) = 3.33** **or** **AB | -3.33 | = 3.33**

4. Summation Symbol (Sums)

- The symbol which is used to denote summation is a Greek letter Sigma ?.

- Let a1, a2, a3, ….. , an be a sequence of numbers. Then the sum a1 + a2 + a3 + ….. + an will be written as:

$$\sum_{j=1}^{n} a_j$$

- where j is called the dummy index or dummy variable. E.g

$$\sum_{j=1}^{n} j = 1 + 2 + 3 + \ldots\ldots + n$$

5. Factorial Function

- n! denotes the product of the positive integers from 1 to n. n! is read as ‘n factorial’, i.e.

  n! = 1 * 2 * 3 * …. * (n-2) * (n-1) *n

- E. g.
  - **4! = 1 * 2 * 3 * 4 = 24**
  - **5! = 1 * 2 * 3 * 4 * 5 = 120**

6. Permutations

- Let we have a set of n elements. A permutation of this set means the arrangement of the elements of the set in some order.
- For example, Suppose the set contains a, b and c. The various permutations of these elements can be:

  abc, acb, bac, bca, cab, cba.

- If there are n elements in the set then there will be n! permutations of those elements.
- It means if the set has 3 elements then there will be 3! = 1 * 2 * 3 = 6 permutations of the elements.

7. Exponents and Logarithms

- Exponent means how many times a number is multiplied by itself. If m is a positive integer, then:
- $a^m = a * a * a * \ldots * a$ (m times) and $a^{-m} = 1 / a^m$
- E.g.
- $2^4 = 2 * 2 * 2 * 2 = 16$
- $2^{-4} = 1 / 2^4 = 1/16$

- The concept of logarithms is related to exponents.
- If b is a positive number, then the logarithm of any positive number x to the base b is written as logbx.
- It represents the exponent to which b should be raised to get x i.e. y = logbx and by = x

E.g.

$\log_2 8 = 3$, Since $2^3 = 8$

$\log_{10}.001 = -3 = 0.001$, since 10^{-3}

$\log_b 1 = 0$, since $b^0 = 1$

$\log_b b = 1$, since $b^1 = b$

Mathematical Functions

FUNCTION	REPRESENTATION	EXAMPLE
FLOOR	$\lfloor x \rfloor$	$\lfloor 7.45 \rfloor = 7 \quad \lfloor -7.45 \rfloor = -8$
CEIL	$\lceil x \rceil$	$\lceil 7.45 \rceil = 8 \quad \lceil -7.45 \rceil = -7$
MODULUS	$k \pmod M = r \quad k = Mq + r \; 0 \le r < M$	$25 \pmod 7 = 4$ $25 \pmod 5 = 0$
INTEGER	INT(X)	INT (3.14) = 3 INT (-8.5) = -8
ABSOLUTE	ABS | X|	ABS | -15| = 15 ABS | 7 | = 7

Mathematical Functions (Cont..)

FUNCTION	REPRESENTATION	EXAMPLE
Factorial	n! = 1 * 2 * 3 * ..... * (n-2) * (n-1) * n	4! = 1 * 2 * 3 * 4 = 24
PERMUTATION	Arrangement of the Elements	n elements arranged in n! ways
EXPONENTS	$a^m,\ a^{-m} = \frac{1}{a^m}, a^{m/n} = \sqrt[n]{a^m} = (\sqrt[n]{a})^m$	2^4 = 2 * 2 * 2 * 2 = 16 2^-4 = 1 / 24 = 1 / 16
LOGARITHM	$y = \log_b n \equiv b^y = n$	$\log_{10} 100 = 2\ as\ 10^2 = 100$
SUMMATION	$\sum_{k=1}^{n} a_k$	$a_1 + a_2 + a_3 + \cdots + a_{n-1} + a_n$

SRM
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University u/s 3 of UGC Act, 1956)

ASYMPTOTIC NOTATIONS
BIG O, OMEGA

Asymptotic Analysis

- The time required by an algorithm falls under three types −
  - **Best Case** − Minimum time required for program execution.
  - **Average Case** − Average time required for program execution.
  - **Worst Case** − Maximum time required for program execution.

Asymptotic Analysis (Cont..)

- Asymptotic analysis of an algorithm refers to defining the mathematical boundation/framing of its run-time performance.
- Derive the best case, average case, and worst case scenario of an algorithm.
- Asymptotic analysis is input bound.
  - Specify the behaviour of the algorithm when the input size increases
- Theory of approximation.
- Asymptote of a curve is a line that closely approximates a curve but does not touch the curve at any point of time.

Asymptotic notations

- Asymptotic notations are mathematical tools to represent the time complexity of algorithms for asymptotic analysis.
- ***Asymptotic order*** is concerned with how the running time of an algorithm increases with the size of the input, if input increases from small value to large values

1. Big-Oh notation (O)
2. Big-Omega notation (Ω)
3. Theta notation (θ)
4. Little-oh notation (o)
5. Little-omega notation (ω)

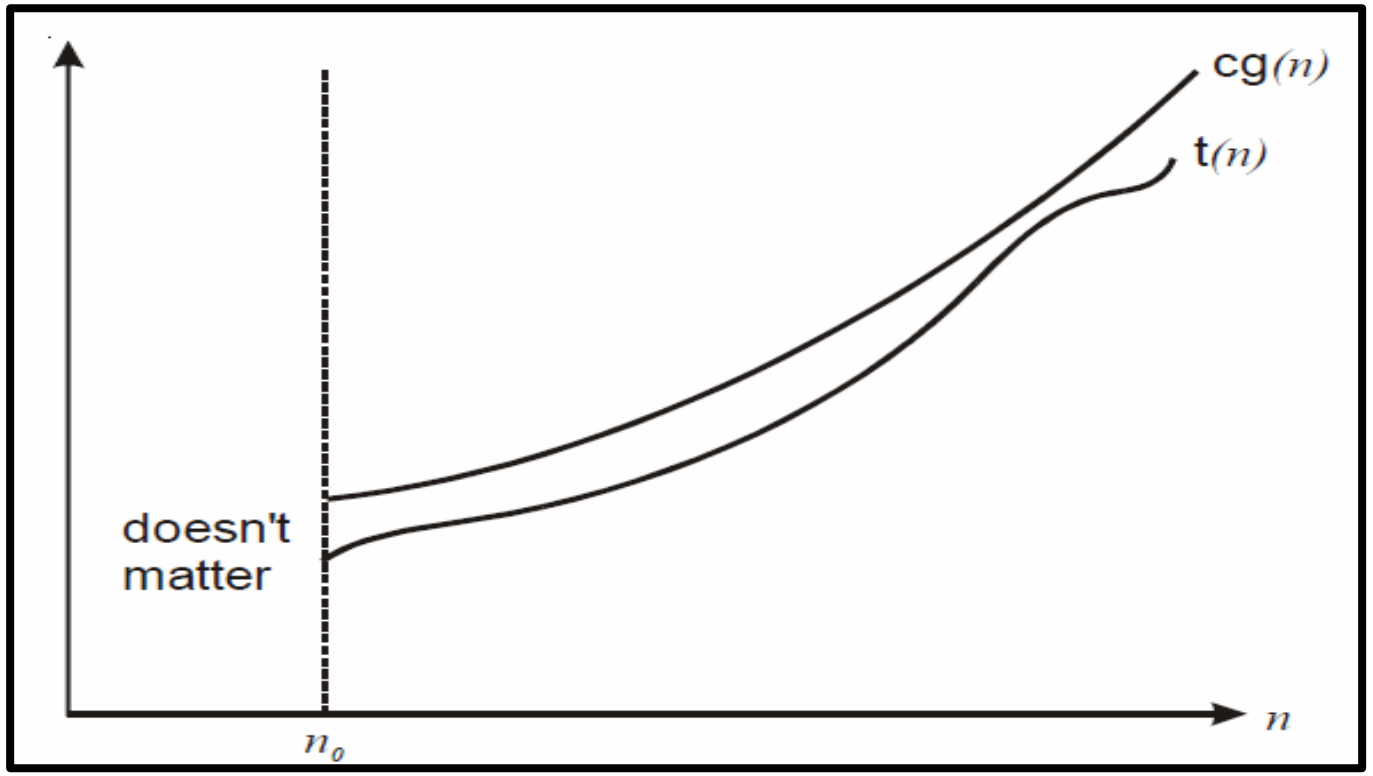
Big-Oh Notation (O)

- Big-oh notation is used to define the worst-case running time of an algorithm and concerned with large values of n.
- ***Definition:*** A function $t(n)$ is said to be in $O(g(n))$, denoted as $t(n) \in O(g(n))$, if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n. i.e., if there exist some positive constant c and some non-negative integer $n0$ such that

$$t(n) \leq cg(n) \text{ for all } n \geq n0$$

- **$O(g(n))$:** Class of functions $t(n)$ that grow no faster than $g(n)$.
- Big-oh puts asymptotic ***upper bound*** on a function.

Big-Oh Notation (O)



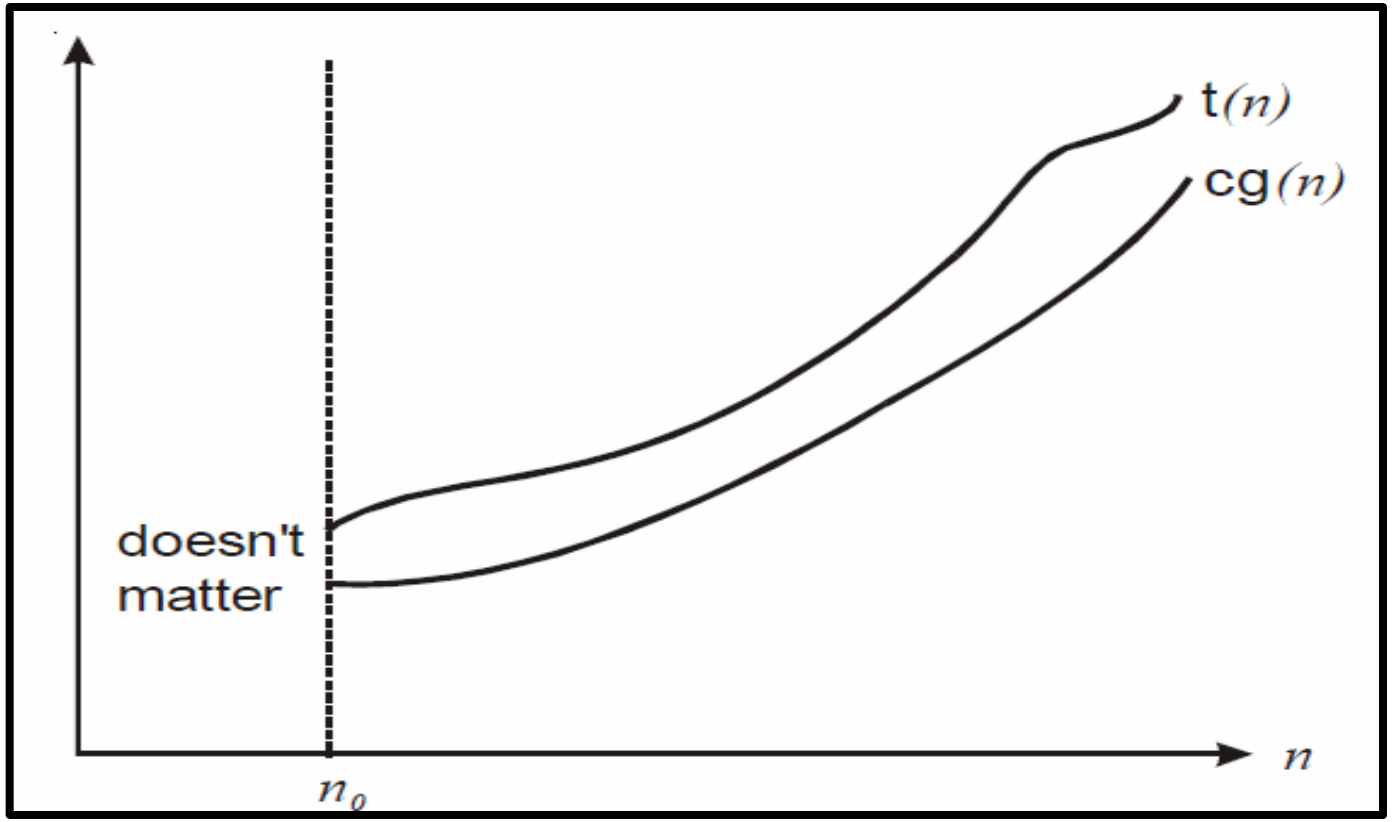
Big-Omega notation (Ω)

- This notation is used to describe the best case running time of algorithms and concerned with large values of n.
- ***Definition:*** A function $t(n)$ is said to be in $\Omega(g(n))$, denoted as $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n. i.e., there exist some positive constant c and some non-negative integer $n0$. Such that

$$t(n) \geq cg(n) \text{ for all } n \geq n0$$

- It represents the ***lower bound*** of the resources required to solve a problem.

Big-Omega notation (Ω)



Big– THETA Θ Notation

f(n) = Θ (g(n)) if and only if there c_1, c_2 and n_0 such that

$$c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$$

for all $n \geq n_0$

$f(n) = O(g(n))$ and $f(n) = \theta(g(n))$

Example: $f(n) = 18n + 9$

If f(n) is between c_1g(n) and c_2g(n), $\forall$ $n \geq n_0$, then f(n) $\in$ Θ(g(n)) and g(n) is an asymptotically tight bound for f(n) and f(n) is amongst h(n) in the set.

Example - THETA

- Show that n2 /2 – 2n = Θ(n2).

Solution By the definition, we can write

$$c_1 g(n) \le h(n) \le c_2 g(n)$$
$$c_1 n^2 \le n^2/2 - 2n \le c_2 n^2$$

Dividing by n^2, we get

$$c_1 n^2/n^2 \le n^2/2n^2 - 2n/n^2 \le c_2 n^2/n^2$$
$$c_1 \le 1/2 - 2/n \le c_2$$

This means $c_2 = 1/2$ because $\lim_{n \to \infty} 1/2 - 2/n = 1/2$ (Big O notation)

To determine c_1 using Ω notation, we can write

$$0 < c_1 \le 1/2 - 2/n$$

We see that $0 < c_1$ is minimum when $n = 5$. Therefore,

$$0 < c_1 \le 1/2 - 2/5$$

Hence, $c_1 = 1/10$

Now let us determine the value of n_0

$$1/10 \le 1/2 - 2/n_0 \le 1/2$$
$$2/n_0 \le 1/2 - 1/10 \le 1/2$$
$$2/n_0 \le 2/5 \le 1/2$$

Example - THETA

$n_0 \geq 5$

You may verify this by substituting the values as shown below.

$c_1 n^2 \leq n^2/2 - 2n \leq c_2 n^2$

$c_1 = 1/10$, $c_2 = 1/2$ and $n_0 = 5$

$1/10(25) \leq 25/2 - 20/2 \leq 25/2$

$5/2 \leq 5/2 \leq 25/2$

Thus, in general, we can write, $1/10n^2 \leq n^2/2 - 2n \leq 1/2n^2$ for $n \geq 5$.

Review Questions

- Indicate constant time complexity in terms of Big-O notation.
  A. O(n)
  B. O(1)
  C. O(logn)
  D. O(n^2)
- Big oh notation is used to describe _____
- Big O Notation is a _____function used in computer science to describe an _____
- Big Omega notation (Ω) provides _____bound.
- Given T1(n) =*O*(f(n)) and T2(n)=*O*(g(n).Find T1(n).T2(n)

Review Questions

1. Which function gives the positive value of the given input?
2. K (mod) M gives the reminder of ____ divided by ____
3. For a given set of number n, the number of possible permutation will be equal to the _____ value of n.
4. _____Notation is used to specify both the lower bound and upper bound of the function.

COMPLEXITY – TIME , SPACE TRADE OFF

SPACE COMPLEXITY

- Space complexity is the **total amount of memory space used by an algorithm/program including the space of input values for execution**. So to find space-complexity, it is enough to calculate the space occupied by the variables used in an algorithm/program.
- Space complexity **S(P)** of any algorithm **P** is,
  - **S(P) = C+SP(I)**
- Where **C** is the fixed part and **S(I)** is the variable part of the algorithm which depends on instance characteristic **I**.

Example:

- Algorithm: SUM(A, B)
- Step 1 - START
- Step 2 - C←A+ B+ 10
- Step 3 – Stop

Here we have three variables A, B & C and one constant. Hence **S(P)=1+3**.
Space requirement depends on data types of given variables & constants. The number will be multiplied accordingly.

How to calculate Space Complexity of an Algorithm?

- In the Example program, 3 integer variables are used.
- The size of the integer data type is 2 or 4 bytes which depends on the architecture of the system (compiler). Let us assume the size as 4 bytes.
- The total space required to store the data used in the example program is 4 +4+4 = 12, Since no additional variables are used, no extra space is required. Hence, **space complexity for the example program is O(1), or constant.**

```
#include<stdio.h>
int main()
{
  int a = 5, b = 5, c;
  c = a + b;
  printf("%d", c);
}
```

EXAMPLE 2

```c
#include <stdio.h>
int main()
{
  int n, i, sum = 0;
  scanf("%d", &n);
  int arr[n];
  for(i = 0; i < n; i++)
  {
    scanf("%d", &arr[i]);
    sum = sum + arr[i];
  }
  printf("%d", sum);
}
```

- In the code given above, the array stores a maximum of n integer elements. Hence, the space occupied by the array is 4 * n. Also we have integer variables such as n, i and sum.
- Assuming 4 bytes for each variable, the total space occupied by the program is **4n + 12** bytes.
- Since the highest order of n in the equation 4n + 12 is n, so **the space complexity is O(n) or linear.**

EXAMPLE 3

```
#include<stdio.h>
int main()
 {
   int a = 5, b = 5, c;
   c = a + b;
   printf("%d", c);
 }
```

Time Complexity

- Time Complexity of an algorithm represents the **amount of time required by the algorithm** to run to completion.
- Time requirements can be defined as a numerical function T(n), where T(n) can be measured as the **number of steps**, provided each step consumes constant time.
- Eg. Addition of two n-bit integers takes n steps.
- Total computational time is T(n) = c*n,
- where **c is the time taken for addition of two bits**.
- Here, we observe that T(n) grows linearly as input size increases.

Time Complexity (Cont..)

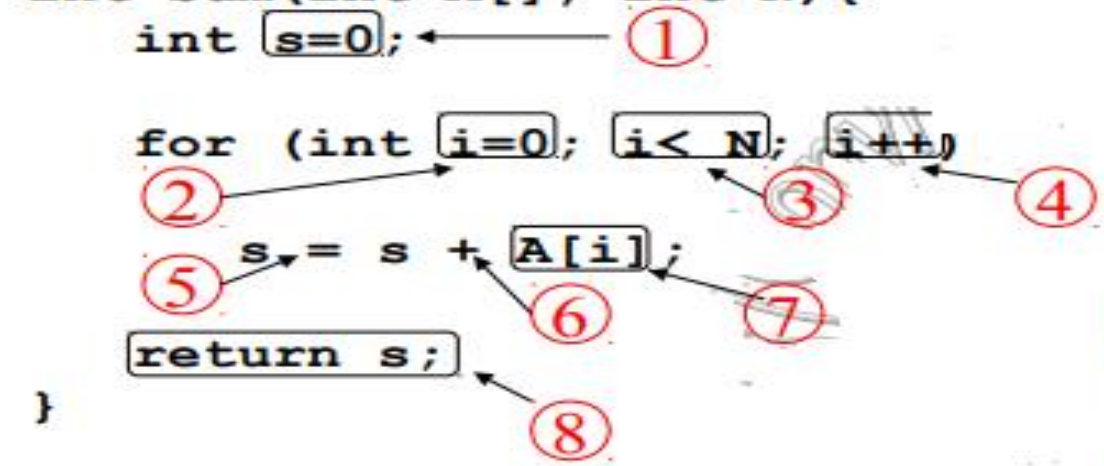
Two methods to find the time complexity for an algorithm

1. Count variable method
2. Table method

Count Variable Method

```
// Input: int A[N], array of N integers
// Output: Sum of all numbers in array A

int Sum(int A[], int N){
    int s=0;
    for (int i=0; i< N; i++)
        s= s + A[i];
    return s;
}
```



1,2,8: Once
3,4,5,6,7: Once per each iteration of for loop, N iteration
Total: 5N + 3
The *complexity function* of the algorithm is : $f(N) = 5N + 3$

Table Method

- The table contains s/e and frequency.
- The s/e of a statement is the amount by which the count changes as a result of the execution of that statement.
- Frequency is defined as the total number of times each statement is executed.
- Combining these two, the step count for the entire algorithm is obtained.

Example 1

Statement		s/e	frequency	total steps
1	**Algorithm** Sum(a, n)	0	–	0
2	{	0	–	0
3	$s := 0.0$;	1	1	1
4	**for** $i := 1$ **to** n **do**	1	$n + 1$	$n + 1$
5	$s := s + a[i]$;	1	n	n
6	**return** s;	1	1	1
7	}	0	–	0
Total				$2n + 3$

Example 3

Consider the following piece of code...

```
int sum(int a, int b)
{
return a+b;
 }
```

In the above sample code, it requires 1 unit of time to calculate a+b and 1 unit of time to return the value. That means, totally it takes 2 units of time to complete its execution. And it does not change based on the input values of a and b. That means for all input values, it requires the same amount of time i.e. 2 units.

If any program requires a fixed amount of time for all input values then its time complexity is said to be Constant Time Complexity

Example 4

	Cost	Times
`i = 1;`	c1	1
`sum = 0;`	c2	1
`while (i <= n) {`	c3	n+1
`  i = i + 1;`	c4	n
`  sum = sum + i;`	c5	n
`}`		

3n+3

Total Cost = c1 +c2 +(n+1)*c3 +n*c4 +n*c5

The time required for this algorithm is proportional to n

Example 5

```
                                    Cost        Times
i=1;                                c1          1
sum = 0;                            c2          1
while (i <= n) {                    c3          n+1
    j=1;                            c4          n
    while (j <= n) {                c5          n*(n+1)
        sum = sum + i;              c6          n*n
        j = j + 1;                  c7          n*n
    }
    i = i +1;                       c8          n
}
```

Total Cost = c1 +c2 +(n+1)*c3 +n*c4 + n*(n+1)*c5+n*n*c6+n*n*c7+n*c8

The time required for this algorithm is proportional to n^2

Review questions

1. Calculate the time complexity for the below algorithms using table method

```
1   Algorithm Fibonacci(n)
2   // Compute the nth Fibonacci number.
3   {
4       if (n ≤ 1) then
5           write (n);
6       else
7       {
8           fnm2 := 0; fnm1 := 1;
9           for i := 2 to n do
10          {
11              fn := fnm1 + fnm2;
12              fnm2 := fnm1; fnm1 := fn;
13          }
14          write (fn);
15      }
16  }
```

2. Determine the frequency counts for all statements in the following two algorithm segments:

```
1  for i := 1 to n do
2      for j := 1 to i do
3          for k := 1 to j do
4              x := x + 1;
```

(a)

```
1  i := 1;
2  while (i ≤ n) do
3  {
4      x := x + 1;
5      i := i + 1;
6  }
```

(b)

3.

```
1  Algorithm Transpose(a, n)
2  {
3      for i := 1 to n − 1 do
4          for j := i + 1 to n do
5          {
6              t := a[i, j]; a[i, j] := a[j, i]; a[j, i] := t;
7          }
8  }
```